

Software Requirements Specification

for

Animal Inspired: Tarantula

Version 1.0

Prepared by

Group Name: Group 17

Batın Kazankaya	2200005985	2200005985@stu.edu.iku.tr
Muhammed Çelik	2400005400	2400005400@stu.edu.iku.tr
Kübra Öyküm Aydın	2200003429	2200003429@stu.edu.iku.tr
Tuba Nazlı İpekyüz	2200004323	2200004323@stu.edu.iku.tr
Huriye Kayadelen	2200005795	2200005795@stu.edu.iku.tr
Damla Günendi	2200004319	2200004319@stu.edu.iku.tr

Instructor:

Prof. Dr. Akhan Akbulut

Course:

COM6064 Software Engineering

Lab Section:

Section D

Teaching Assistant:

Büşra Kocaçınar

Date:

19.05.2026

Contents

1 Introduction	1
1.1 Project Purpose and Scope, and Objectives	3
1.1.1 Product Backlog	5
1.2 Roles and responsibilities	6
1.3 Technical Assumptions and Constraints	8
1.3.1 Supported Hardware Platforms	8
1.3.2 Version Control	9
1.3.3 Programming Languages	9
1.3.4 Operating System Support	10
1.3.5 Development Tools and Libraries	10
1.3.6 Communication Protocols	11
1.3.7 System, Memory, and Performance Constraints	11
1.3.8 System Limitations	12
1.3.9 Technical Assumptions	12
1.3.10 Design Standards and Safety	13
1.4 Naming Conventions	13
1.4.1 Coding Standards	13
1.4.2 Documentation and SRS Standards	15
2 Requirements	1
2.1 Functional Requirements	1
2.2 Non Functional Requirements	2
2.2.1 Performance Requirements	2
2.2.2 Safety and Security Requirements	3
2.2.3 Software Quality Attributes	4
2.2.3.1 Reliability	4
2.2.3.2 Maintainability	4
2.2.3.3 Portability	4
2.2.3.4 Usability	5
2.2.3.5 Robustness	5
3 Other Requirements	6
4 System Architecture and Architectural Design	8
4.1 Logical View	8
4.2 Deployment View	11
4.3 Use Case View	12
4.3.1 Use Case Scenarios	13
5 Design and Implementation	14
Physical Prototype	14

Master STM32 Firmware (main.c)	20
USART Configuration.....	20
Non-Blocking Command Queue Architecture	20
BNO055 Stability Fix	21
locomotion.h - Tripod Gait Library	21
Activity Diagram - UML Command Flow.....	22
C# WPF Control Panel (TarantulaControl).....	24
Raspberry Pi Zero 2W - QR Perception Script	24
Unity 3D Digital Twin Visualization	25
Developer Guide for Future Work	25
6 Other Supporting Information	26
Sample Firmware Code Blocks.....	26
7 References	32

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Batın Kazankaya, Tuba Nazlı İpekyüz, Kübra Öyküm Aydın, Huriye Kayadelen, Muhammed Çelik, Damla Günendi	Initial SRS document. Introduction, purpose, scope, objectives, product backlog, roles and responsibilities, technical assumptions and constraints, naming conventions, functional requirements, non-functional requirements, system architecture, logical view, deployment view, use case view, design and implementation, and supporting information sections are completed. FR-13 Video Streaming & Perception Feedback requirement is added to Table 2.1.	19.05.2026

1 Introduction

The Tarantula Hexapod Robot System is a robotic system developed to demonstrate the design, control, communication, and visualization of a six-legged mobile robot inspired by the walking structure of a tarantula. The project combines embedded systems, servo motor control, sensor integration, wireless communication, autonomous navigation logic, and Unity-based digital twin visualization within a single integrated system. The main purpose of the project is not only to make a physical robot walk, but also to create a complete robotic platform that can sense its environment, receive target information, communicate with external systems, and represent its movement in a virtual 3D environment. Robotic systems have become an important part of modern engineering because they are used in many areas such as industrial automation, search and rescue operations, surveillance, education, defense technologies, medical systems, and scientific research. Among different types of mobile robots, legged robots have a special importance because they can move on surfaces where wheeled robots may have difficulty. A hexapod robot, which has six legs, provides a more stable walking structure compared to two-legged or four-legged robots. Since three legs can remain on the ground while the other three legs move, the robot can maintain balance more easily during movement. This makes hexapod robots suitable for experimental studies about locomotion, balance, motion planning, servo control, and autonomous robotic behavior. In the past, most mobile robot studies focused on simple movement mechanisms, manual control, and basic obstacle avoidance. However, current robotic systems are expected to include more advanced features such as autonomous decision making, sensor-based orientation, wireless telemetry, visual target recognition, and digital simulation. For this reason, the Tarantula Hexapod Robot System was designed as a multi-module system rather than a simple walking robot. The system includes an STM32-based embedded control unit, PCA9685 servo driver infrastructure, BNO055 orientation sensor, GPS module, Raspberry Pi camera-based QR target detection, LoRa communication, slave robot communication, and Unity 3D digital twin visualization. These components work together to create a more complete and demonstrable robotic system. The physical movement of the robot is controlled by the STM32 microcontroller. The STM32 side is responsible for low-level movement control, walking sequence execution, servo motor coordination, sensor reading, communication processing, and autonomous navigation decisions. Servo motors are driven through the PCA9685 servo driver module in order to control multiple joints in a stable and organized way. The walking structure of the robot is based on coordinated leg movement. The robot uses locomotion functions such as standing up, stepping forward, stepping backward, turning left, turning right, and stopping. These functions allow the robot to perform both manual and autonomous movement operations. The robot also includes a BNO055 IMU sensor for orientation measurement. This sensor provides yaw, pitch, and roll values, which are important for understanding the direction and attitude of the robot. In the autonomous navigation logic, the yaw value is especially important because the robot must compare its current heading with the target direction. The firmware also checks the calibration status of the sensor. This makes the system more reliable because autonomous movement should not begin before the orientation sensor is sufficiently ready. If the sensor reading fails repeatedly, the software attempts to reinitialize the I2C connection and the BNO055 sensor, which improves system robustness. GPS is another important part of the system. The GPS module provides the current position of the robot by sending NMEA messages. The software parses GGA-type GPS messages and extracts latitude, longitude, altitude, satellite count, and GPS fix status. These values are used for both telemetry reporting and autonomous navigation. The robot can calculate the distance between its

current location and the target coordinate by using the Haversine formula. It can also calculate the required bearing angle toward the target. After this calculation, the robot compares the target bearing with its current yaw angle and decides whether it should turn left, turn right, or move forward. The project also includes QR-based target acquisition. A Raspberry Pi camera system is used to read target coordinate information from a QR code. When the STM32 receives a QR message from the Raspberry Pi, it stores the target latitude and longitude values as the active destination. After that, the master robot forwards this information to the ground station and also sends the target coordinates to the slave robot. This structure makes the system more advanced because the target is not hard-coded into the firmware. Instead, the target can be updated dynamically through a QR-based visual input mechanism. Wireless communication is provided through LoRa. LoRa communication is used for sending telemetry data, warnings, acknowledgements, target information, and slave robot commands. The master robot communicates with two different external nodes: the ground station and the slave robot. The ground station receives robot status information such as GPS data, BNO055 orientation values, calibration information, warning messages, and arrival messages. The slave robot receives target coordinates and control commands from the master robot. This master-slave communication structure allows the system to support coordinated multi-robot behavior. The autonomous navigation logic is one of the main functional parts of the project. Autonomous mode can only start when the system has a valid target coordinate, GPS fix, and sufficient BNO055 calibration level. This prevents the robot from starting autonomous movement without the required information. When autonomous mode is active, the robot continuously checks the updated GPS position, calculates the distance to the target, calculates the bearing angle, and compares this angle with the current yaw value. If the robot is not aligned with the target direction, it turns left or right. If the direction error is within the defined threshold, the robot moves forward. When the robot reaches the target area within the defined arrival distance, the system stops autonomous navigation and sends an arrival message to the ground station. In addition to the physical robot, the project includes a Unity 3D Digital Twin Visualization module. This module provides a virtual representation of the Tarantula hexapod robot in a 3D campus-style environment. The Unity model is structured with separate leg parts so that coxa and femur joint movements can be visualized. The Unity module can receive manual movement commands from the C# WPF control panel through TCP, including forward, backward, left, right, and stop commands. It can also display leg motion by using incoming SERVO: angle data when available, or a demonstration gait when servo data is not available. The movement logic was updated so that the virtual robot moves according to its local facing direction after turning. In addition, a drone-style camera follow system keeps the robot visible during movement, and the Unity camera output is streamed into the C# WPF interface through a dedicated Unity View panel. This Software Requirements Specification document describes the software requirements, system behavior, architectural structure, functional requirements, non-functional requirements, design decisions, hardware and software interfaces, and supporting information for the Tarantula Hexapod Robot System. The document focuses on the complete system from the software point of view, including embedded firmware, communication structure, sensor integration, movement control, autonomous navigation, and Unity-based visualization. The purpose of this SRS is to clearly define what the system should do, how the main modules should interact with each other, and what constraints must be considered during development and testing. Overall, the Tarantula Hexapod Robot System aims to provide a realistic and educational robotic platform that demonstrates the integration of mechanical movement, embedded software, sensor-based navigation, wireless communication, and 3D digital visualization. The project is valuable because it connects theoretical engineering concepts with practical implementation. It allows the development team to work on real hardware, movement algorithms, communication protocols,

sensor data processing, and software architecture. At the same time, the Unity digital twin layer improves real-time observation, system monitoring, testing, and visual understanding of the robot behavior.

1.1 Project Purpose and Scope, and Objectives

The purpose of the Tarantula Hexapod Robot System is to design and develop a six-legged robotic platform that can demonstrate stable walking behavior, embedded control, sensor-based navigation, wireless communication, target acquisition, and digital visualization within a single integrated system. The project aims to go beyond a basic manually controlled robot by combining physical locomotion with intelligent software modules and a Unity-based digital twin environment. In this way, the project provides both a functional robotic prototype and a demonstrable software system that can be used for observation, testing, presentation, and future development. The system is inspired by the walking structure of a tarantula and uses a hexapod robot design. The six-legged structure was selected because it provides better balance and stability compared to robots with fewer legs. During movement, some legs can remain on the ground while the other legs move, allowing the robot to maintain support and reduce the risk of falling. This makes the robot suitable for studying coordinated multi-legged movement, servo motor control, walking algorithms, balance behavior, and motion optimization. The project therefore focuses not only on making the robot move, but also on making the movement controlled, repeatable, and suitable for further improvement. The scope of this project includes the embedded software running on the STM32 microcontroller, the communication structure between hardware modules, the processing of sensor data, the execution of walking commands, the autonomous navigation logic, the master-slave robot communication, and the Unity 3D Digital Twin Visualization module. The STM32-based master controller is responsible for controlling the movement of the physical robot, reading sensor data, receiving commands, sending telemetry, processing QR-based target coordinates, and deciding movement actions during autonomous navigation. The Unity side is responsible for visualizing the robot in a three-dimensional environment and supporting the final demonstration of the system. One of the main purposes of the project is to implement a reliable embedded control structure for the robot. The STM32 controller manages the low-level behavior of the robot by coordinating the locomotion functions. These functions include standing up, moving forward, moving backward, turning left, turning right, and stopping. The movement system is based on servo motor control through the PCA9685 servo driver module. Since a hexapod robot requires several servo motors to move its legs, the servo driver is used to organize PWM-based control signals and make the system more manageable. This structure allows the robot to perform controlled leg movements instead of uncontrolled or random motor actions.

Another important purpose of the system is to support sensor-based movement decisions. The robot uses the BNO055 IMU sensor to obtain orientation values such as yaw, pitch, and roll. These values help the system understand the robot's current direction and attitude. In autonomous navigation, yaw information is used to compare the robot's current heading with the target direction. The system also checks the calibration state of the BNO055 sensor before starting autonomous movement. This is important because navigation decisions should be made only when orientation data is reliable enough. The use of sensor calibration control improves the safety and correctness of the robot's autonomous behavior. The project also includes GPS-based localization. The GPS module provides the current geographical position of the robot by sending NMEA data. The software extracts latitude, longitude, altitude, satellite count, and fix status from the GPS messages. These values are used to determine

whether the robot has a valid position and to calculate the distance and direction to the target. The robot uses the Haversine formula to calculate the distance between the current coordinate and the target coordinate. It also calculates the bearing angle to determine the direction in which the robot should move. Therefore, GPS data is a key part of the autonomous navigation scope of the project. A further purpose of the project is to provide dynamic target acquisition through a QR-based system. Instead of defining the destination manually inside the code, the robot can receive target coordinates from a Raspberry Pi camera module. The Raspberry Pi reads QR code data and sends the coordinate information to the STM32 through UART communication. After receiving this data, the STM32 stores the coordinates as the active target, forwards the information to the ground station, and transmits the same target to the slave robot. This design makes the system more flexible because the target can be changed without modifying the firmware. The scope also includes LoRa-based wireless communication. The master robot communicates with the ground station and the slave robot through LoRa. The ground station receives telemetry data, acknowledgement messages, warning messages, QR target information, and arrival status. The slave robot receives target coordinates and control commands from the master robot. This communication structure allows the project to demonstrate not only a single mobile robot but also a coordinated robotic system. The master-slave structure improves the project scope by adding distributed communication and multi-robot coordination features. The autonomous navigation objective of the system is to allow the robot to move toward a QR-defined target coordinate using GPS and IMU data. Autonomous mode can only start when three main readiness conditions are satisfied: a valid target coordinate must be available, GPS fix must be active, and the BNO055 calibration level must be sufficient. When these conditions are met, the robot calculates the distance to the target and the direction angle. If the yaw error is greater than the allowed threshold, the robot turns left or right to correct its direction. If the robot is aligned with the target direction, it moves forward. When the robot reaches the target area within the predefined arrival distance, autonomous movement stops and an arrival message is sent to the ground station.

The Unity 3D Digital Twin Visualization module is another major part of the project scope. This module provides a virtual representation of the Tarantula hexapod robot in a 3D campus-style environment. The Unity scene helps the robot structure, movement behavior, and system operation to be observed more clearly from the control interface. The virtual robot model is organized with separate leg parts so that coxa and femur joint movements can be represented visually.

The Unity module is connected with the C# WPF control panel through TCP communication. The interface can send manual movement commands such as forward, backward, left, right, and stop to the Unity scene. The movement logic was updated so that the virtual robot moves according to its local facing direction instead of a fixed world direction. Therefore, when the robot turns right or left and then receives a forward command, it moves in the direction it is currently facing. This allows direction-based and diagonal movement in the Unity visualization.

The Unity module can also display leg motion by using incoming SERVO: angle data when available. When servo angle data is received, Unity maps the incoming 12 angle values to the corresponding virtual leg joints. If servo data is not available, a demonstration gait can be used to keep the leg movement visible during testing. In addition, a drone-style camera follow system keeps the robot visible while it moves, and the Unity camera output is streamed into the C# WPF interface through a dedicated Unity View panel. The scope of this SRS document is mainly software-focused. It describes the requirements of the embedded firmware, communication logic, sensor integration, navigation algorithm, command processing, telemetry transmission, and Unity digital twin visualization. Mechanical design details such as the physical body shape, leg materials, and exact assembly process

are not the primary focus of this document unless they directly affect software behavior. However, servo angle calibration, walking stability, locomotion timing, and leg coordination are considered within the scope because they are directly related to the robot's software-controlled movement. The project has several main objectives. The first objective is to develop a stable embedded control system that can manage the robot's locomotion and hardware interfaces. The second objective is to implement coordinated hexapod movement using servo motors and predefined walking functions. The third objective is to integrate GPS and BNO055 sensor data into the robot's navigation process. The fourth objective is to receive target coordinates dynamically through a Raspberry Pi QR system. The fifth objective is to establish LoRa communication between the master robot, ground station, and slave robot. The sixth objective is to implement a basic autonomous navigation behavior based on distance and bearing calculations. The seventh objective is to create a Unity-based digital twin environment for visualization and presentation. The expected benefit of the project is to create a complete and educational robotic system that demonstrates the relationship between hardware, embedded software, communication, sensor processing, navigation, and simulation. The system allows developers and observers to see how different engineering components work together. It also provides a strong foundation for future improvements such as obstacle detection, more advanced gait algorithms, real-time Unity synchronization, improved slave robot coordination, environmental mapping, and autonomous decision-making. Therefore, the project is valuable both as a working robotic prototype and as a learning platform for robotics and software engineering. In conclusion, the Tarantula Hexapod Robot System aims to provide a modular, expandable, and demonstration-ready robotic platform. Its purpose is not limited to physical walking; it also includes target detection, navigation, communication, telemetry, and digital visualization. The project scope covers the complete software behavior required to operate and present the robot as an integrated system. The objectives of the project support both current implementation and future development, making the system suitable for academic evaluation, engineering practice, and further research in mobile robotics.

1.1.1 Product Backlog

The product backlog of the Tarantula Hexapod Robot System is divided into five main development units: Mechanical Development, Electronics & Hardware, Embedded Software Development, Communication & Navigation, and Unity 3D Digital Twin Visualization. Each development unit has its own to-do list, which defines the tasks and responsibilities that must be completed for the successful design, implementation, integration, testing, and demonstration of the project. The backlog was prepared according to the main modules of the robot system and supports the development of both the physical robot and the virtual visualization environment.

Table 1.1 Product Backlog

Development Unit	To-Do-List
Mechanical Development	The Mechanical Development unit focuses on the physical structure of the Tarantula Hexapod Robot. In this unit, the six-legged tarantula-inspired robot body is designed and assembled. The mechanical structure is prepared to support stable standing and walking movements. The leg joints are designed according to hexapod movement requirements, and each leg is arranged so that it can move without colliding with the body or with other legs. The physical placement of servo motors is also planned carefully on the robot body. In addition, the body balance is optimized so that the robot can remain stable while multiple legs are supporting the structure during movement. The mechanical connection points of the coxa and femur joints are adjusted, and the overall structure is improved for forward, backward, left-turn, and right-turn movements. The durability of the

Development Unit	To-Do-List
	robot body is checked during repeated walking tests, and mechanical corrections are made according to servo angle tests and real walking behavior.
Electronics & Hardware	The Electronics & Hardware unit includes the integration of the STM32 microcontroller, PCA9685 servo driver, servo motors, BNO055 IMU sensor, GPS module, LoRa module, and Raspberry Pi camera system. UART connections are prepared for Raspberry Pi, GPS, and LoRa communication, while I2C is used for the BNO055 sensor. Power distribution and hardware connections are checked to prevent unstable sensor readings, communication errors, or servo power problems.
Embedded Software Development	The Embedded Software Development unit focuses on developing the STM32 master robot firmware. This includes configuring UART, I2C, and GPIO interfaces, implementing the command queue, developing manual movement commands, and connecting locomotion functions with servo control. The firmware also parses GPS data, reads BNO055 orientation and calibration values, generates telemetry messages, and supports both manual and autonomous operation modes.
Communication & Navigation	The Communication & Navigation unit includes the message structure and autonomous navigation logic of the robot. The system receives QR-based target coordinates from Raspberry Pi, stores them as the active target, forwards them to the ground station, and sends them to the slave robot through LoRa. It also calculates distance and bearing using GPS data, compares the target direction with BNO055 yaw data, and decides whether the robot should turn or move forward. Autonomous mode starts only when target data, GPS fix, and IMU calibration are ready.
Unity 3D Digital Twin Visualization	The Unity 3D Digital Twin Visualization unit focuses on creating the virtual representation of the Tarantula robot in a 3D campus-style environment. A drone-style camera follow system is developed to observe the robot clearly during movement. The Unity module receives movement commands from the C# WPF interface through TCP, moves the robot according to its local facing direction after turning, displays leg movement using SERVO: angle data or a demonstration gait, and streams the Unity camera view into the interface.
Testing & Calibration	The Testing & Calibration unit includes servo testing, coxa and femur angle calibration, standing balance tests, walking and turning tests, GPS and BNO055 tests, QR target reception tests, LoRa communication tests, telemetry tests, autonomous navigation tests, and Unity visualization tests. Full system integration tests are performed before the final demonstration, and detected problems are corrected before submission.

1.2 Roles and responsibilities

Table 1.2 Member Index Matrix

MemberID	Member StdID	Member Name
M1	2200004323	Tuba Nazlı İpekyüz
M2	2200003429	Kübra Öyküm Aydın
M3	2200005795	Huriye Kayadelen
M4	2200005985	Batın Kazankaya
M5	2400005400	Muhammed Çelik

MemberID	Member StdID	Member Name
M6	2200004319	Damla Günendi

Table 1.2 Member Index Matrix shows indexes for each member of the group for the simplicity of the Table 1.3 Responsibility Assignment Matrix, demonstrating role and responsibilities in the project lifecycle.

Table 1.3 Responsibility Assignment Matrix

MemberID	Role	Responsibility
Tuba Nazlı İpekyüz	Embedded Software, Walking Algorithm & Servo Control Engineer	Responsible for embedded software development, servo control, and locomotion behavior. Develops and tests movement functions such as standing up, moving forward, moving backward, turning left, turning right, and stopping. Works on servo angle calibration, gait timing, walking stability, and movement optimization for the hexapod robot.
Kübra Öyküm Aydın	Electronics & Hardware Engineer	Responsible for the electrical and electronic hardware part of the system. Works on the integration of electronic components, wiring structure, voltage regulation, power distribution, and electrical safety. Supports the connection of STM32, PCA9685 servo driver, servo motors, BNO055 IMU sensor, GPS module, LoRa module, and Raspberry Pi-related hardware. Ensures that the electronic system is stable enough for testing and final demonstration.
Huriye Kayadelen	Unity 3D Digital Twin Developer	Responsible for the Unity 3D Digital Twin Visualization module. Designs the Unity scene, places the Tarantula robot model in a campus-style environment, and develops the drone-style camera follow system. Also connects Unity with the C# WPF interface through TCP commands, supports local-direction robot movement, and enables the Unity camera stream to be shown inside the interface.
Batın Kazankaya	LoRa Communication & User Interface Developer	Responsible for LoRa-based wireless communication and user interface development. Works on communication between the master robot, ground station, and slave robot. Supports telemetry transmission, QR target forwarding, command transfer, acknowledgement messages, warning messages, and arrival messages. Also develops the interface side that allows the operator to monitor robot data, communication status, target information, and system feedback during operation.
Muhammed Çelik	Embedded Software, Walking Algorithm & Servo Control Engineer	Responsible for LoRa-based wireless communication between system modules. Works on communication between the master robot, ground station, and slave robot. Supports telemetry transmission, QR target forwarding, command transfer, acknowledgement messages, warning messages, and arrival messages. Ensures that communication data is correctly transferred during manual and autonomous operation.

MemberID	Role	Responsibility
Damla Günendi	Raspberry Pi / QR Detection Engineer	Responsible for embedded software development, servo control, and locomotion behavior. Develops and tests movement functions such as standing up, moving forward, moving backward, turning left, turning right, and stopping. Works on servo angle calibration, gait timing, walking stability, and movement optimization for the hexapod robot.

1.3 Technical Assumptions and Constraints

The development of the Tarantula Hexapod Robot System has several technical assumptions and constraints that define the hardware environment, software tools, communication protocols, programming languages, operating systems, and design limitations used throughout the project lifecycle. Since the project includes a physical hexapod robot, embedded control software, sensor-based navigation, wireless communication, Raspberry Pi-based QR target detection, user interface support, and Unity 3D Digital Twin Visualization, the system must be designed according to the capabilities and limitations of each component.

1.3.1 Supported Hardware Platforms

The system is designed to run on a multi-module hardware architecture consisting of the following main components:

- **STM32 Microcontroller:**

Responsible for the main embedded control logic of the robot. It manages locomotion commands, servo control flow, GPS data processing, BNO055 sensor reading, LoRa communication, target coordinate handling, telemetry transmission, and autonomous navigation decisions.

- **PCA9685 Servo Driver Module:**

Used for controlling multiple servo motors through PWM signals. Since the robot has several servo motors for leg movement, PCA9685 is used to provide more organized and stable servo control.

- **Servo Motors:**

Used for moving the robot legs. The servo motors are responsible for coxa and femur joint movements and allow the robot to stand, walk forward, walk backward, turn left, turn right, and stop.

- **BNO055 IMU Sensor:**

Used for obtaining orientation data such as yaw, pitch, and roll. The yaw value is especially important for autonomous navigation because it is used to compare the robot's current heading with the target direction.

- **GPS Module:**

Used for obtaining the robot's current latitude, longitude, altitude, satellite count, and GPS fix status. GPS data is used for telemetry and autonomous navigation.

- **LoRa Communication Module:**

Used for wireless communication between the master robot, ground station, and slave robot. It supports telemetry transmission, QR target forwarding, command transfer, warning messages, acknowledgement messages, and arrival messages.

- Raspberry Pi Camera System:

Used for QR-based target coordinate detection. The Raspberry Pi reads the QR code and sends latitude-longitude information to the STM32 master controller through UART communication.

- Ground Station / User Interface Device:

Used for monitoring robot data, receiving telemetry, observing communication status, and displaying system feedback during testing and demonstration.

- PC / Laptop for Unity Visualization:

Used to run the Unity 3D Digital Twin Visualization module and display the robot in a virtual campus-style environment.

1.3.2 Version Control

- Git:

Used for version control, source code management, and tracking project changes during development.

- GitHub:

May be used as a remote repository platform for storing firmware files, Unity project files, interface files, documentation, and project-related resources.

1.3.3 Programming Languages

- Embedded C / C:

Used for STM32 firmware development. The embedded code manages sensor reading, UART communication, I2C communication, LoRa packet processing, command queue handling, telemetry generation, and autonomous navigation logic.

- C# for Unity:

Used for Unity 3D Digital Twin Visualization scripts. Unity scripts are responsible for camera follow behavior, robot visualization, scene interaction, and future real-time data communication support.

- Python:

Used or assumed for Raspberry Pi QR detection operations. Python can be used to process camera frames, detect QR codes, extract coordinate data, and send target information to STM32 through serial communication.

- C# / Desktop Interface Development:

Used for the ground station or user interface side if the monitoring interface is developed as a desktop application. The interface can display telemetry, warning messages, target information, communication status, and robot feedback.

1.3.4 Operating System Support

Development and testing are mainly performed on Windows-based computers. The embedded firmware, Unity project, and interface-related development are expected to be prepared and tested primarily on Windows 10 or Windows 11 systems.

Table 1.4 OS Requirements

OS	Version	Architectures	Supported For
Windows 10	Client Version 1607+	x64, x86, Arm64	STM32 development, Unity development, C# WPF, documentation
Windows 11	Version 22000+	x64, x86, Arm64	STM32 development, Unity development, C# WPF, documentation
Raspberry Pi OS	Bookworm (64-bit)	ARM64	QR detection, camera processing, UART communication
Linux	Compatible distribution	x64 / ARM	Optional testing for Python-based QR or serial scripts

The main development environment is assumed to be Windows 10/11. Raspberry Pi OS is required for the Raspberry Pi camera and QR detection module. Python-based scripts may also run on Linux systems if compatible libraries and drivers are installed.

1.3.5 Development Tools and Libraries

- **STM32CubeIDE:**

Used for STM32 firmware development, project configuration, HAL library usage, compilation, flashing, and debugging.

- **STM32 HAL Library:**

Used for configuring and controlling UART, I2C, GPIO, clock settings, interrupts, and other microcontroller peripherals.

- **Unity Editor:**

Used for creating the 3D Digital Twin Visualization module, virtual environment, robot model placement, camera follow behavior, and demonstration scene.

- **Visual Studio / Visual Studio Code:**

Used for editing Unity C# scripts, interface code, Python scripts, and general project files.

- **Python 3.x:**

Used for Raspberry Pi QR detection and camera-based target coordinate processing.

- **Serial Communication Libraries:**

Used for data transfer between Raspberry Pi and STM32 or between external systems and Unity/interface components.

- **LoRa Module Configuration Tools:**

Used for configuring LoRa communication parameters such as address, channel, baud rate, and message transmission behavior.

- **3D Modeling / CAD Tools:**

Used for preparing or adjusting robot parts, servo placements, body structure, and possible 3D-printed components.

1.3.6 Communication Protocols

- UART Communication:

Used between STM32 and Raspberry Pi for QR target coordinate transfer. It is also used for GPS data reception and LoRa module communication in the embedded system.

- I2C Communication:

Used between STM32 and BNO055 IMU sensor. The STM32 reads yaw, pitch, roll, and calibration data from the BNO055 sensor through I2C.

- LoRa Wireless Communication:

Used for communication between the master robot, ground station, and slave robot. It supports telemetry data, target coordinate forwarding, command messages, acknowledgement messages, warning messages, and arrival messages.

- Serial Data Communication:

Used or planned for communication between the physical robot and external visualization or monitoring systems.

- TCP Communication:

Supported as a future communication option for Unity or external interface command transfer.

1.3.7 System, Memory, and Performance Constraints

- The STM32 microcontroller has limited memory and processing resources; therefore, firmware must be optimized and unnecessary operations should be avoided.

- Servo control must be executed with stable timing because incorrect delays or sudden movements may cause unstable walking behavior.

- GPS data accuracy depends on satellite visibility, signal quality, and fix status. Autonomous navigation should not start without a valid GPS fix.

- BNO055 orientation values depend on calibration quality. Autonomous navigation should not start if the sensor calibration level is not sufficient.

- LoRa communication bandwidth is limited; therefore, telemetry and command messages should be short, structured, and transmitted at reasonable intervals.

- Telemetry transmission is expected to be periodic and lightweight. In the current firmware logic, GPS and BNO055 telemetry data are sent at defined time intervals.

- The robot's movement speed is limited by servo motor torque, mechanical balance, leg structure, and power capacity.

- Unity visualization performance depends on the computer hardware, scene complexity, number of objects, lighting, and camera behavior.

- Raspberry Pi QR detection performance depends on camera quality, lighting conditions, QR code visibility, and processing speed.

1.3.8 System Limitations

- The robot does not use wheel odometry because it is a legged robot.
- The robot does not include advanced obstacle detection in the current scope.
- The autonomous navigation system depends mainly on GPS coordinates and BNO055 yaw data.
- GPS-based navigation may be less accurate in indoor environments or areas with weak satellite signal.
- The robot operates best on flat or semi-flat surfaces because leg movement stability is affected by ground conditions.
- Servo motor torque and battery capacity limit the robot's payload and walking duration.
- The Unity Digital Twin module is mainly used as a visualization layer and does not replace STM32-based low-level control.
- Real-time synchronization between the physical robot and Unity is considered a future improvement unless fully implemented during the project.
- LoRa communication can be affected by distance, antenna quality, environmental obstacles, and interference.
- QR detection requires the QR code to be visible to the Raspberry Pi camera under acceptable lighting conditions.

1.3.9 Technical Assumptions

- The STM32 microcontroller is assumed to be the main controller of the physical robot.
- The PCA9685 servo driver is assumed to provide stable PWM control for the robot's servo motors.
- The BNO055 sensor is assumed to provide usable yaw, pitch, and roll data after calibration.
- The GPS module is assumed to provide valid NMEA GGA messages when satellite fix is available.
- The Raspberry Pi camera system is assumed to detect QR codes containing target latitude and longitude values.
- The Raspberry Pi and STM32 are assumed to communicate through UART.
- The LoRa modules are assumed to be configured with compatible address, channel, and baud rate settings.
- The ground station or interface is assumed to receive telemetry and system messages from the robot.

- The Unity environment is assumed to run on a computer capable of supporting 3D visualization.
- The robot is assumed to be tested in an environment suitable for legged movement.

1.3.10 Design Standards and Safety

- All electronic components must operate within safe voltage and current ranges.
- Servo motor power lines should be arranged carefully to prevent voltage drops and unstable movement.
- Sensor and communication wiring should be organized to reduce connection errors and signal instability.
- The system should prevent autonomous mode from starting when target data, GPS fix, or IMU calibration is not ready.
- The robot should be tested first in manual mode before autonomous mode is activated.
- The robot should be placed on a safe and open test surface during walking tests.
- Emergency stop or stop command behavior should be available during operation.
- Modular design principles should be followed so that hardware and software modules can be tested, replaced, or improved independently.
- The Unity visualization module should be separated from low-level embedded control to avoid unsafe direct control behavior.
- Documentation, naming conventions, and code organization should be kept consistent throughout the project.

1.4 Naming Conventions

1.4.1 Coding Standards

The Tarantula Hexapod Robot System follows consistent naming conventions and coding standards in order to make the embedded software, Unity visualization module, Raspberry Pi QR module, LoRa communication, and user interface easier to understand, test, and maintain.

- Programming Paradigm
 - o Mostly imperative and modular programming is used in the STM32 embedded firmware.
 - o Event-driven programming is used for UART interrupt-based communication.
 - o Object-oriented programming is used in Unity C# scripts.
 - o Script-based programming is used for Raspberry Pi QR detection.
- Variable and Function Naming
 - o Descriptive function names are used to clearly indicate the purpose of each function.
 - o Module-based function naming is used in STM32 embedded C code:
 - BNO055_Init()
 - BNO055_Read()

- GPS_ParseNMEA()
- SendTelemetry()
- RelayToGS()
- SendToSlave() o Movement-related functions are named according to the robot action:
- Hexapod_StandUp()
- Hexapod_StepForward()
- Hexapod_StepBackward()
- Hexapod_TurnLeft()
- Hexapod_TurnRight()
- Hexapod_Stop() o snake_case is used for local and global variables in STM32 embedded firmware:
- my_lat
- my_lon
- target_lat
- target_lon
- has_target
- bno_yaw
- bno_pitch
- bno_roll o ALL_CAPS_WITH_UNDERSCORES is used for constants and macros:
- BNO055_ADDR
- ARRIVED_DIST_M
- YAW_THRESHOLD
- MIN_CAL_LEVEL
- TELEM_INTERVAL
- CMD_QUEUE_SIZE o Descriptive naming is followed to indicate the function, sensor, or communication purpose:
- GPS_ParseNMEA()
- BNO055_Read()
- SendTelemetry()
- SendToSlave()

- Communication Message Naming

o Structured prefixes are used for communication messages:

- \$M: for master robot messages
- \$M:GPS for GPS telemetry messages
- \$M:BNO for BNO055 telemetry messages
- \$QR: for QR target messages received from Raspberry Pi
- \$T: for target coordinate messages sent to the slave robot

- Unity Naming

o PascalCase is used for Unity C# class names and scene objects:

- CameraFollowController
- DigitalTwinManager
- TarantulaRobotModel
- CampusBuilding

▪MainCamera o camelCase is used for Unity script variables:

- cameraHeight
- backwardDistance
- lookAheadDistance
- rotationOffset

- Raspberry Pi Script Naming

o snake_case is used for Python script file names and variables:

- qr_reader.py
- target_sender.py
- target_latitude
- target_longitude

1.4.2 Documentation and SRS Standards

1. Section titles, table names, figure names, diagrams, and references follow the structure of the given SRS template.
2. Tables are numbered according to their order in the document:
 - a. Table 1.1 Product Backlog
 - b. Table 1.2 Member Index Matrix
 - c. Table 1.3 Responsibility Assignment Matrix
 - d. Table 1.4 OS Requirements

3. Requirement identifiers are written with short prefixes:
 - a. Functional Requirements: FR-1, FR-2, FR-3
 - b. Non-Functional Requirements: NFR-1, NFR-2, NFR-3
 - c. Performance Requirements: PR-1, PR-2, PR-3
 - d. Safety Requirements: SR-1, SR-2, SR-3
4. Consistent terminology is preserved throughout the document. Terms such as **Tarantula Hexapod Robot, STM32, PCA9685, BNO055, GPS, LoRa, Raspberry Pi, Ground Station, Slave Robot, and Unity Digital Twin** are used in the same form.
5. Abbreviations are written in uppercase form:
 - a. GPS
 - b. IMU
 - c. UART
 - d. I2C
 - e. PWM
 - f. QR
6. The document uses formal technical language and avoids informal expressions. All module names, requirement names, and communication message names are written consistently to improve readability and maintainability.

2 Requirements

2.1 Functional Requirements

Functional requirements capture the intended behavior of the system. They are divided into six functional areas covering locomotion, sensor integration, communication, autonomous navigation, coordination, perception feedback, and the user interface.

Table 2.1 Functional Requirements

ID	Functional Area	Requirement
FR-1	Locomotion Control	The system shall execute tripod gait movement using the locomotion library: Hexapod_StandUp(), Hexapod_StepForward(), Hexapod_StepBackward(), Hexapod_TurnLeft(), Hexapod_TurnRight(), Hexapod_Stop(). 12 MG90S servos are controlled through PCA9685 channels 0-11 at 50 Hz PWM.
FR-2	Non-Blocking Command Queue	The system shall implement a circular command buffer of CMD_QUEUE_SIZE=4. UART ISR pushes single-byte commands without HAL_Delay. Main loop pops and dispatches when loco_busy=0. Each step triggers \$M:ACK upon completion.
FR-3	BNO055 IMU Sensing	The system shall read BNO055 NDOF Euler angles (yaw, pitch, roll) via I2C1 at 100 kHz. Calibration status (sys_cal, gyro_cal, accel_cal, mag_cal) is read from register 0x35. I2C recovery (DeInit -> Init -> BNO055_Init) triggers after 3 consecutive HAL_ERROR returns. Zero-guard prevents overwriting last valid reading with all-zeros data.
FR-4	GPS Sensing	The system shall parse BN-880 NMEA GGA sentences (GPGGA/GNGGA) via USART2 at 115200 baud. Extracted values: latitude, longitude (degree+decimal minute to decimal degree conversion), altitude, satellite count, fix status. GPS fix (my_fix=1) is required before enabling autonomous mode.
FR-5	LoRa Communication	The system shall use Ebyte E220 fixed-point addressing mode. Each packet is prefixed with 3-byte header [AddrH, AddrL, Channel]. GS node buffers bytes until newline and forwards complete packets to C# panel. UART ORE flag is cleared in HAL_UART_ErrorCallback and Receive_IT re-armed.
FR-6	QR Target Acquisition	The system shall receive \$QR:lat,lon packets from Raspberry Pi Zero 2W via USART1 at 9600 baud. Master stores target_lat and target_lon, sets has_target=1, relays \$QR: to GS for C# display, and dispatches \$T:lat,lon to Slave robot.
FR-7	Autonomous Navigation	Autonomous mode shall activate only when has_target=1, my_fix=1, and bno_sys_cal >= MIN_CAL_LEVEL (1). Haversine distance and Bearing computed each GPS update. If distance < ARRIVED_DIST_M (2.0 m): stop + send \$M:ARR. If yaw_err > YAW_THRESHOLD (10 deg): turn left or right. Otherwise: step forward.

ID	Functional Area	Requirement
FR-8	Inter-Robot Coordination	Master shall automatically dispatch \$T:lat,lon to Slave upon QR decode. Slave independently navigates to same target using own GPS and BNO055. This constitutes the required functional coordination: Master state directly affects Slave behaviour.
FR-9	Telemetry Transmission	Master and Slave shall transmit \$M:GPS / \$\$:GPS and \$M:BNO / \$\$:BNO packets at TELEM_INTERVAL (1000 ms) when loco_busy=0. GS relays all packets to C# panel.
FR-10	Emergency Stop	Emergency stop command ('S') shall immediately call Hexapod_Stop(), drain command queue, set autonom_mode=0, and relay 'S' to Slave robot. No pending steps shall execute after stop.
FR-11	Unity Digital Twin	The Unity scene shall include the Tarantula robot model in a campus-style 3D environment. The Unity module shall receive movement commands from the C# WPF interface through TCP and support forward, backward, left, right, and stop commands. The virtual robot shall move according to its local facing direction after turning. The CameraFollowController shall smoothly follow the robot during movement. The Unity camera output shall be streamed and displayed inside the C# WPF interface through a dedicated Unity View panel.
FR-12	C# WPF Control Panel	TarantulaControl panel shall connect to GS via USB serial at 9600 baud; send directional commands (F/B/L/R/S/G/A); display \$M:GPS and \$M:BNO telemetry; show Slave state (IDLE/WALK/DONE); maintain ACK flow control with 3-second watchdog; save log to .txt file; send TCP commands to the Unity Digital Twin; and display the Unity camera stream in a dedicated Unity View panel.
FR-13	Video Streaming & Perception Feedback	The Raspberry Pi Zero 2W shall host a Flask-based HTTP server to stream real-time camera frames processed by OpenCV. The system shall draw bounding boxes around detected QR codes for operator visualization on the C# WPF panel and implement a local state log (detected_qrs.txt) to guarantee that only unique target coordinates are transmitted via USART1, preventing communication pipeline congestion.

2.2 Non Functional Requirements

2.2.1 Performance Requirements

Table 2.2 Performance Requirements

ID	Metric	Requirement
PR-1	Command Response Time	C# button press to servo movement start: < 200 ms under normal LoRa indoor conditions.
PR-2	Telemetry Latency	GPS and BNO055 data updated in C# panel within < 500 ms of generation at Master.

ID	Metric	Requirement
PR-3	Step Cycle Time	One complete tripod gait cycle (forward step) shall complete in ≤ 700 ms.
PR-4	LoRa Packet Loss	Packet loss rate $< 5\%$ over 10-minute continuous telemetry session at indoor range (< 50 m).
PR-5	GPS Navigation Accuracy	Robot shall arrive within 2 m of QR-decoded GPS target in open-field conditions.
PR-6	BNO055 Heading Accuracy	Yaw error ≤ 15 deg after NDOF calibration ($\text{sys_cal} \geq 2$). Autonomous navigation corrections within ± 10 deg threshold.
PR-7	I2C Stability	Zero I2C bus lockouts during continuous 1-hour operation at 100 kHz with loco_busy serialization.

2.2.2 Safety and Security Requirements

Table 2.3 Risk Analysis - Safety and Security Requirements

RiskID	Risk Definition	Priority
SR-1	Servo power brownout resets STM32 during high-torque movement. XL4015 5 V rail must be electrically isolated from STM32 3.3 V logic supply.	High
SR-2	HAL_Delay inside ISR freezes main loop after first command. All motor control must execute in main loop only; ISR performs only CmdQueue_Push().	High
SR-3	I2C bus lockout from concurrent BNO055 + PCA9685 access. loco_busy flag must serialize I2C access between telemetry reads and locomotion writes.	High
SR-4	Emergency stop ('S') must drain command queue and call Hexapod_Stop() immediately regardless of loco_busy state.	High
SR-5	LoRa E220 in transparent mode passes address headers as data, corrupting all command bytes. Fixed-point mode must be configured via AT commands before deployment.	High
SR-6	BNO055 reinit loop (mode polling) continuously zeros Euler data. OPR_MODE register polling must be removed from BNO055_Read(). Mode set once in BNO055_Init().	Medium
SR-7	Autonomous mode activation without GPS fix or target causes undefined navigation. has_target, my_fix, and bno_sys_cal $\geq$ MIN_CAL_LEVEL must all be true.	Medium
SR-8	UART ORE (overrun error) permanently locks RX interrupt.	Medium

RiskID	Risk Definition	Priority
	HAL_UART_ErrorCallback must clear ORE flag and re-arm HAL_UART_Receive_IT.	
SR-9	loco_busy stuck > 3 seconds causes permanent UI lockout. Watchdog timer in C# resets _locoBusy after 3 s if no ACK received.	Medium
SR-10	RPi QR partial packet transmission if GPS parsing fails. Transmission only occurs after successful lat/lon validation.	Low

2.2.3 Software Quality Attributes

2.2.3.1 Reliability

- UART overrun error recovery in HAL_UART_ErrorCallback ensures RX interrupt is never permanently lost across all three UART ports.
- I2C hardware recovery (DeInit -> 10 ms delay -> MX_I2C1_Init -> 10 ms delay -> BNO055_Init) is triggered after 3 consecutive read failures.
- The 3-second watchdog timer in the C# WPF panel prevents permanent UI lockout if ACK is lost due to LoRa packet drop.
- The loco_busy flag prevents telemetry I2C reads from racing with PCA9685 I2C writes on the shared I2C1 bus.

2.2.3.2 Maintainability

- All servo PWM constants (PWM_MIN, PWM_MAX, COXA_FORWARD, FEMUR_STAND, etc.) are defined as macros in locomotion.h for single-point calibration.
- LoRa node addresses are defined as byte arrays at file scope (addr_gs[], addr_slave[]), so address changes require only one-line edits.
- The command processing switch-case in the main loop is self-documenting and easily extended with new single-byte command codes.
- The C# ParseLine() function is table-driven by string prefix; adding new telemetry types requires one new else-if branch.

2.2.3.3 Portability

- STM32 firmware uses only HAL library functions and is portable to any STM32F4 series device with minor CubeMX pin reconfiguration.
- locomotion.h is self-contained with no external dependencies and can be dropped into any STM32 project targeting PCA9685 via I2C.
- The C# WPF application targets .NET 8 and runs on Windows 10/11 x64 without additional runtime installation.
- Raspberry Pi Python script dependencies (pyzbar, picamera2, pyserial) are available on Raspberry Pi OS versions from Bullseye onwards.

2.2.3.4 Usability

- The C# panel provides clear visual feedback: green CONNECTED / red DISCONNECTED, progress bar for step count, and colour-coded Slave state (grey IDLE, blue WALK, green DONE).
- Keyboard shortcuts (W/A/S/D/Space) allow single-hand control panel operation.
- BNO055 calibration warning is displayed when sys_cal=0 to prevent user confusion about heading quality.

2.2.3.5 Robustness

- Autonomous mode is guarded by three independent readiness flags (has_target, my_fix, bno_sys_cal >= MIN_CAL_LEVEL). A single failure blocks autonomous activation.
- The loco_busy watchdog force-resets the main loop if it is stuck for more than 3 seconds, preventing permanent locomotion freeze.

3 Other Requirements

This section defines additional requirements covering system factors, performance measures, and the electronics and power budget of the Tarantula Hexapod Robot System.

Table 3.1 System Factors & Variables

Factors	Variables
LoRa Signal Quality	Packet Loss Rate / RSSI
GPS Satellite Count	Position Fix Availability / Navigation Accuracy
Battery Voltage	Servo Torque Output / System Power Stability
I2C Bus Load	BNO055 Read Latency / PCA9685 Write Jitter
BNO055 Calibration Status	Heading Accuracy / Navigation Reliability
Environmental Terrain	Step Success Rate / Gait Stability

Table 3.2 Performance Metrics

Performance Metric	Definition
Command Success Rate	$\geq 98\%$ of C# commands result in observable robot movement within 200 ms.
Telemetry Continuity	GPS and BNO055 data updated in C# panel at ≥ 1 Hz during active missions.
Navigation Arrival Accuracy	Robot arrives within 2 m of QR-decoded GPS target in open-field test.
Coordination Trigger Reliability	Master-to-Slave \$T\$: dispatch occurs within 1 s of Master arrival (\$M:ARR) in 100% of tests.
Step Cycle Consistency	Standard deviation of tripod gait cycle time < 50 ms over 100 consecutive steps.
I2C Error Rate	< 1 I2C HAL_ERROR per minute during combined locomotion + telemetry operation.
LoRa Packet Loss	$< 5\%$ packet loss over 10-minute continuous telemetry session at indoor range.

Electronics Requirements & Power Budget per robot:

- 12 x MG90S servo motors @ 0.3 A stall = 3.6 A @ 5 V $\rightarrow$ 18 W peak servo load
- STM32F401CCU6 + logic peripherals approximately 150 mA @ 3.3 V $\rightarrow$ 0.5 W logic
- BN-880 GPS approximately 50 mA, BNO055 approximately 12 mA, Ebyte E220 TX approximately 120 mA
- Total peak per robot approximately 3.93 A @ 5 V = 19.65 W
- Recommended battery: 2S LiPo 2200 mAh $\rightarrow$ approximately 20 minutes continuous locomotion

Table 3.3 Costs

Component	Amount	Price	Total Price
MG90S Servo	24	80,88	1941,12
PCA9685	2	189,41	378,82

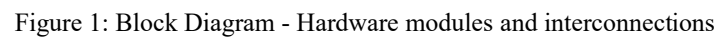
Component	Amount	Price	Total Price
BNO055	1	2803,84	2803,84
L86-M33 GPS	1	520,17	520,17
E22-900T22D	3	525,74	1577,22
STM32F401	3	226,47	679,41
Raspberry Pi Zero 2 W	1	933,86	933,86
Raspberry Pi Zero 2 W Camera	1	838,48	838,48
Aspilsan 18650	4	104,42	417,68
Total			10.090,6

4 System Architecture and Architectural Design

The Tarantula Hexapod Robot System uses a three-nodeLoRa network architecture with a desktop control interface. The system is structured in four layers: User Interface Layer (C# WPF panel), Communication Bridge Layer (GS STM32), Perception and Control Layer (Master and Slave STM32), and Actuation Layer (PCA9685 + servo motors). The architecture follows modular design principles where each component has a clearly defined interface. The communication chain is: C# WPF Panel → USB CDC (9600 baud) → GS STM32 (USART2) → LoRa E220 (USART6) → Master STM32 (address 0x0002). Master STM32 also communicates with: Raspberry Pi Zero 2W (USART1, 9600 baud), BN-880 GPS (USART2, 115200 baud), BNO055 + PCA9685 (I2C1, 100 kHz), Slave STM32 (LoRa address 0x0003).

4.1 Logical View

The logical view shows the internal structure of the system. The Block Diagram (Figure 1) presents all hardware modules and their interconnections. The Layered Architecture Diagram (Figure 2) shows the four-layer software structure. Each layer communicates only with adjacent layers, maintaining separation of concerns.



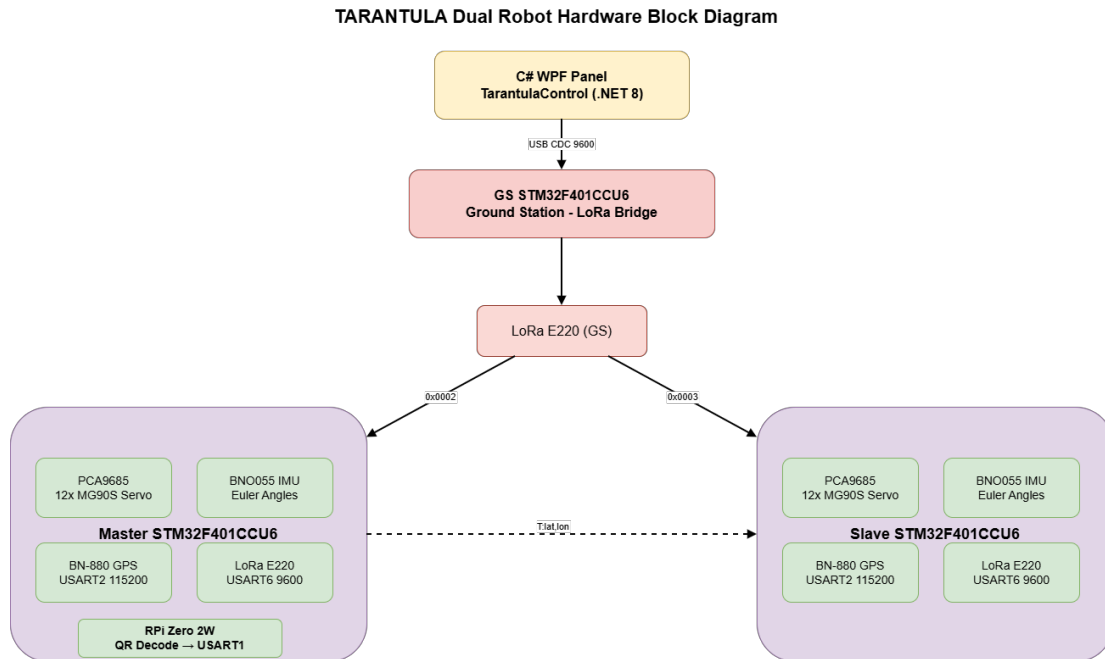


Figure 2: Layered Architecture Diagram - Four-layer software structure

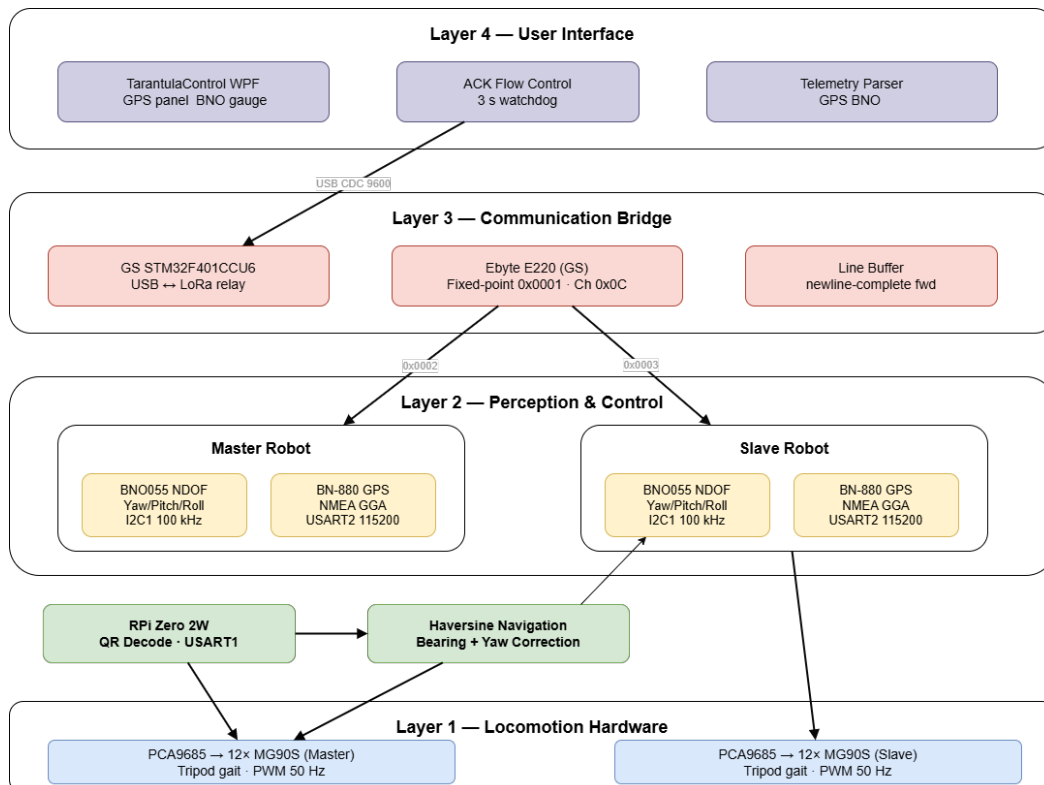
TARANTULA Layered Architecture Diagram — Four-Layer Software Structure

Figure 3: System Context Diagram - External actors and system boundary

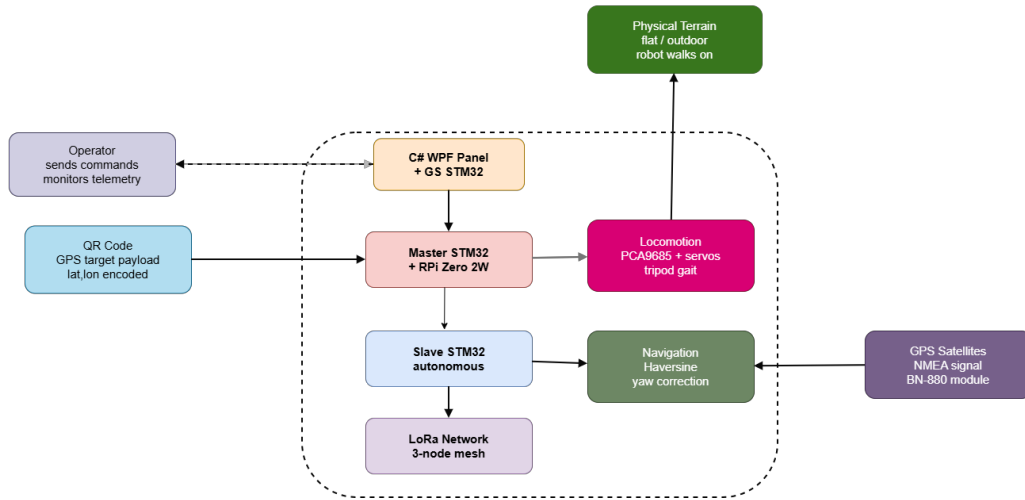


Figure 4: SOA Diagram - Service-oriented architecture of each node

4.2 Deployment View

The deployment architecture consists of three physical tiers. The Robot Hardware Tier includes two hexapod robots, each containing STM32F401CCU6, PCA9685, 12 MG90S servos, BNO055, BN-880 GPS, Ebyte E220 LoRa, and XL4015 power regulator. The Master robot additionally carries a Raspberry Pi Zero 2W. The Ground Station Tier contains a single STM32F401CCU6 with USB connection to the operator PC and LoRa E220 for wireless bridging. The Operator PC Tier runs the TarantulaControl WPF application on Windows 10/11, communicating with the GS STM32 via USB CDC serial port. A separate PC optionally runs the Unity 3D Digital Twin Visualization module.

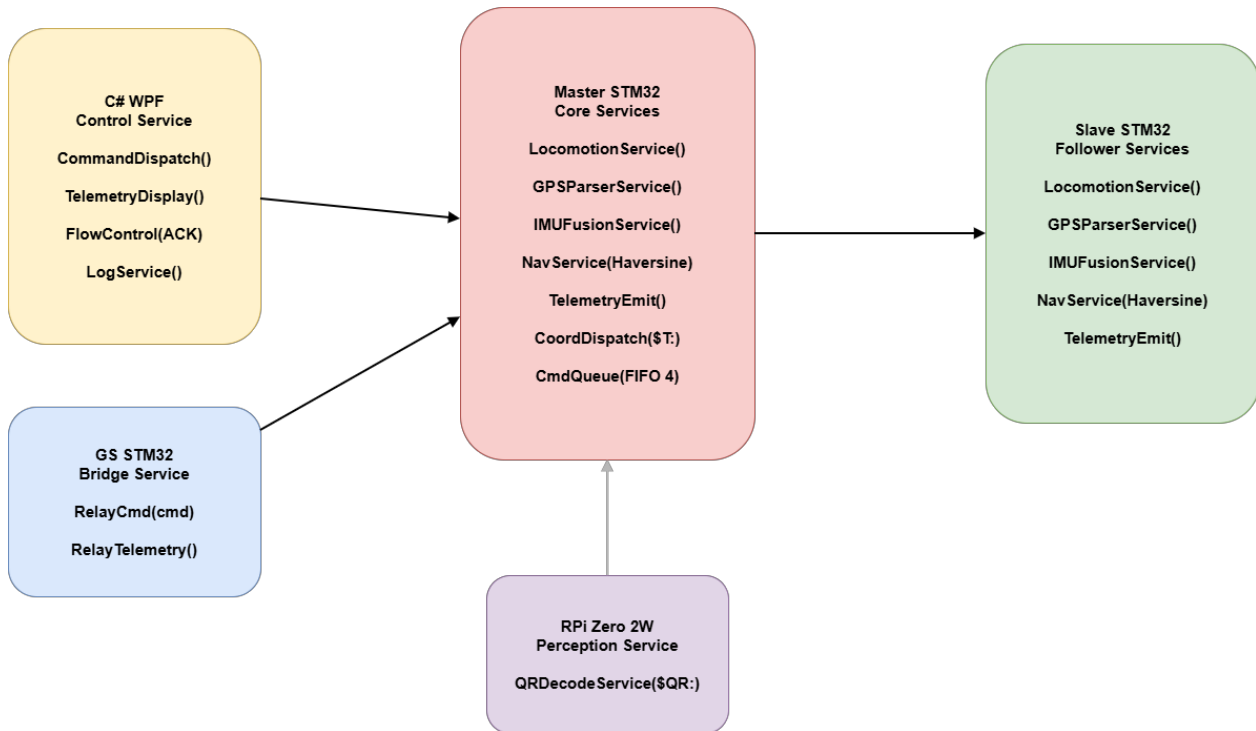


Figure 5: Deployment Diagram - Physical node distribution

4.3 Use Case View

The Use Case Diagram (Figure 6) shows four main actors and eleven use cases. The Operator actor interacts with the system through the C# WPF control panel and sends directional commands, monitors telemetry, and triggers autonomous mode. The Master STM32 actor executes locomotion, GPS/IMU sensing, QR reception, navigation, and coordination. The Slave STM32 actor receives targets and navigates autonomously. The Raspberry Pi Zero 2W actor decodes QR codes and dispatches target coordinates.

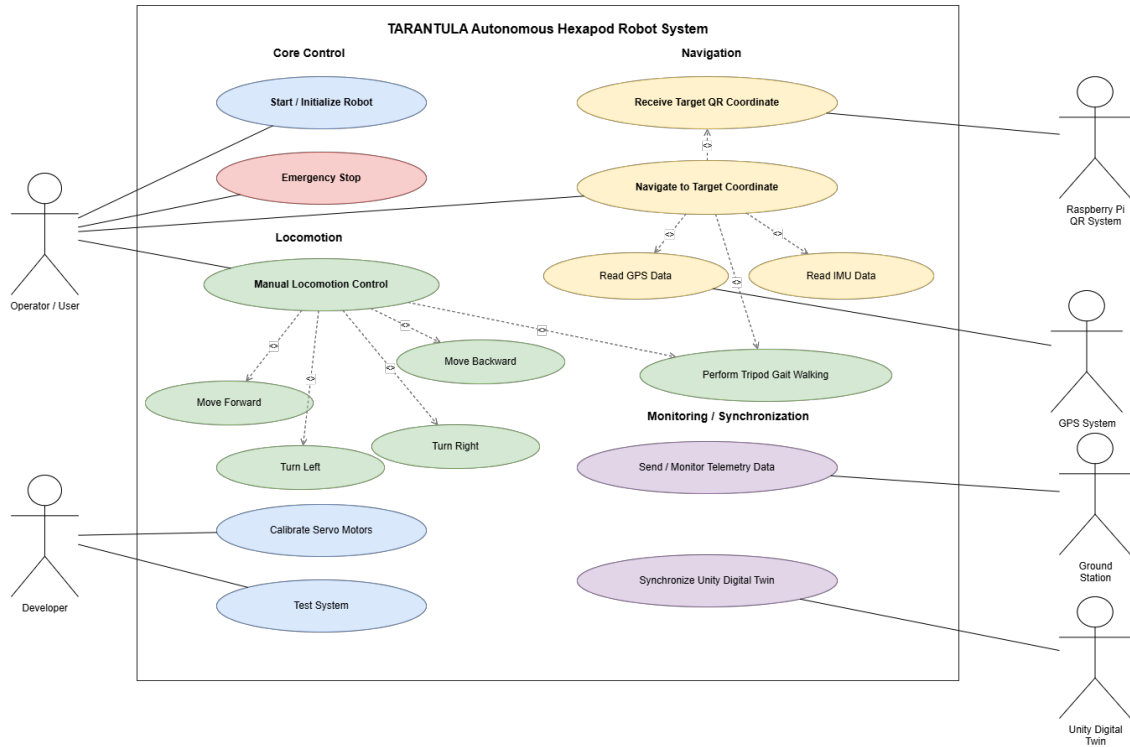


Figure 6: Use Case Diagram - System actors and use cases

4.3.1 Use Case Scenarios

Table 4.1 Use Case Scenarios

Use Case	Pre-Conditions	Main Conditions	Post-Conditions
Manual Locomotion Control	GS STM32 connected to C# panel. Master firmware running.	Operator presses directional button on C# panel. GS relays command byte to Master over LoRa. Master executes one tripod gait step. Master sends \$M:ACK.	C# panel re-enables directional buttons.
QR Target Acquisition	RPi Zero 2W running QR script. Camera has view of QR code.	RPi decodes QR GPS payload, transmits \$QR:lat,lon. Master stores target, relays to GS, dispatches \$T: to Slave.	C# panel displays QR target. Slave stores target for navigation.
Master Autonomous Navigation	has_target=1, my_fix=1, bno_sys_cal>=1. Operator sends 'G'.	Master computes Haversine bearing each GPS update. Steps forward if aligned (yaw_err<10 deg), turns if not. Telemetry at 1 Hz.	Master arrives within 2 m -> stops -> sends \$M:ARR -> C# shows arrival.
Slave Autonomous Navigation	Slave received \$T: from Master. Slave GPS fix obtained.	Slave computes Haversine independently. Navigates to same GPS target. Transmits \$\$:GPS, \$\$:BNO telemetry.	Slave arrives -> C# panel Slave state -> DONE (green).
Emergency Stop	Any locomotion mode active.	Operator presses STOP or Space. C# sends 'S'. GS relays. Master calls Hexapod_Stop(), relays to Slave.	All servo movement halts. Command queue cleared. Both robots in standing position.

5 Design and Implementation

This section provides guidelines and implementation details for the Tarantula Hexapod Robot System, including the embedded firmware, locomotion library, communication system, Raspberry Pi QR module, C# WPF interface, and Unity Digital Twin module.

Physical Prototype

Figures 11-13 show the physical Tarantula hexapod robots built for this project. Two identical units were constructed: a black prototype used during early servo calibration and gait algorithm development, and a blue production unit used for all integration, navigation, and coordination tests. Both robots share the same hardware architecture described in the Technical Assumptions and Constraints section.



Figure 11: Tarantula hexapod robot (black prototype) during assembly. All six legs equipped with MG90S servos are visible, and the wiring harness connecting the PCA9685 PWM driver board to the servo motors is exposed beneath the dome-shaped body enclosure. This prototype was used for initial servo calibration and gait testing.

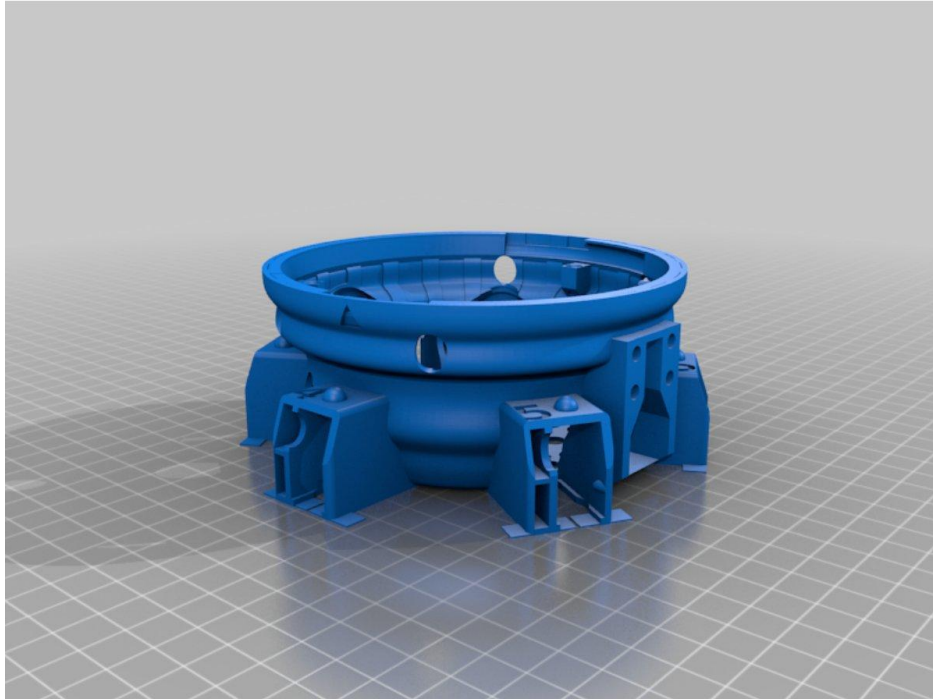


Figure 11b: 3D CAD render of the Tarantula hexapod robot base/turret ring assembly showing the central body frame structure and leg mounting brackets.



Figure 12: Completed blue Tarantula hexapod robot standing on the test surface. The latticed dome body enclosure houses the STM32F401CCU6 Black Pill microcontroller, PCA9685 servo driver, BNO055 IMU, BN-880 GPS module, and Ebyte E220-900T22D LoRa transceiver. Black rubber end-tips on the legs provide traction during tripod gait locomotion.

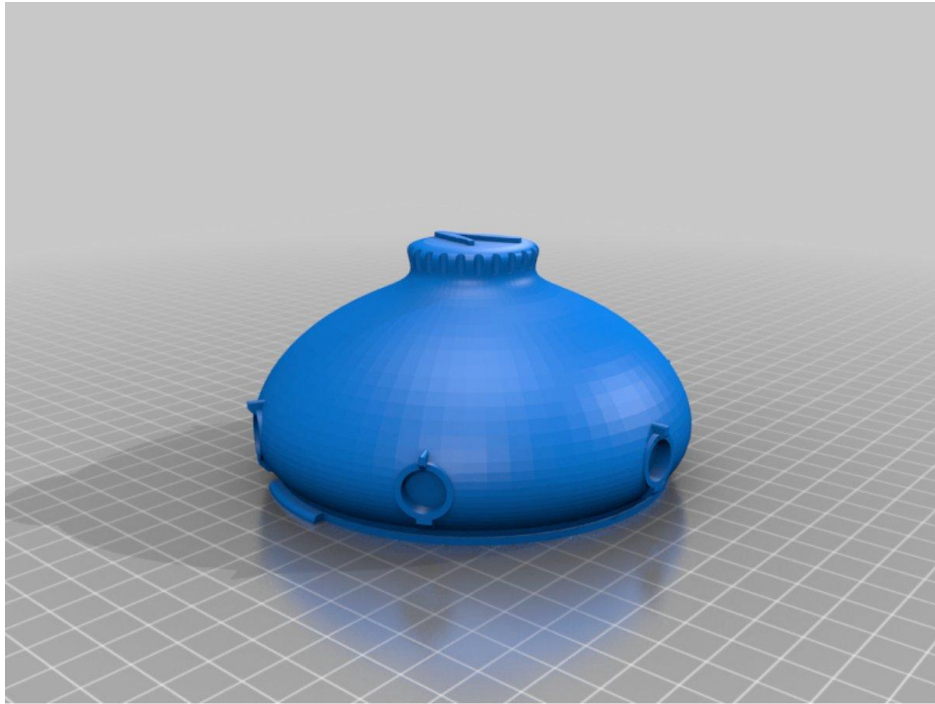


Figure 12b: 3D CAD render of the dome-shaped body enclosure showing the gear ring at the top, sensor port openings, and mounting clips around the perimeter.



Figure 13: Top-down view of the Tarantula hexapod showing the six-leg radial arrangement (FR, MR, RR, FL, ML, RL). Each leg is independently controlled by two MG90S servos - Coxa (horizontal rotation) and Femur (vertical lift) - mapped to PCA9685 channels 0-11. The MG90S servo labels are clearly visible on the coxa joints in this view.

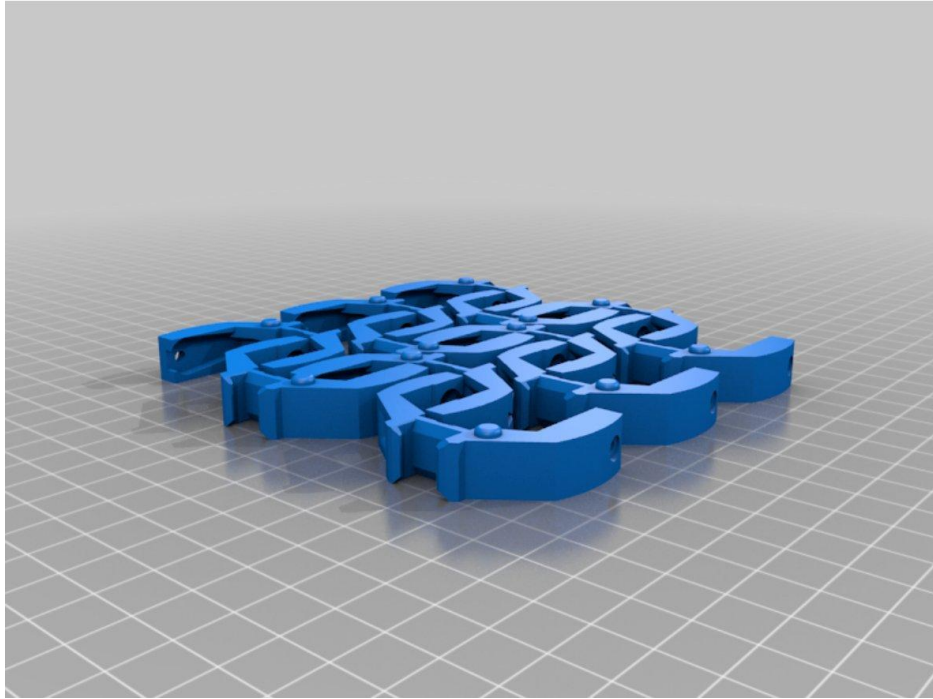


Figure 13b: 3D CAD render showing the top-down view of the six-leg radial arrangement. The latticed body structure and coxa joint positions for all six legs (FR, MR, RR, FL, ML, RL) are clearly visible.

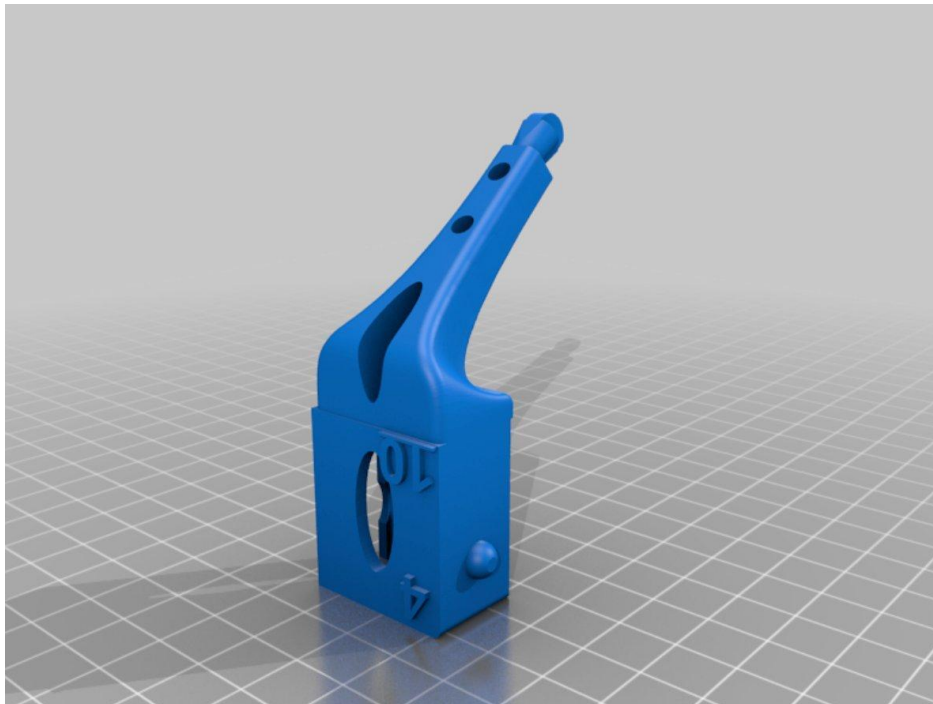


Figure 13c: 3D CAD render of a single leg assembly showing the coxa-femur joint structure. The femur arm extends upward with ball-end tip. Component label "4" and servo channel "10" are embossed on the coxa mount block.

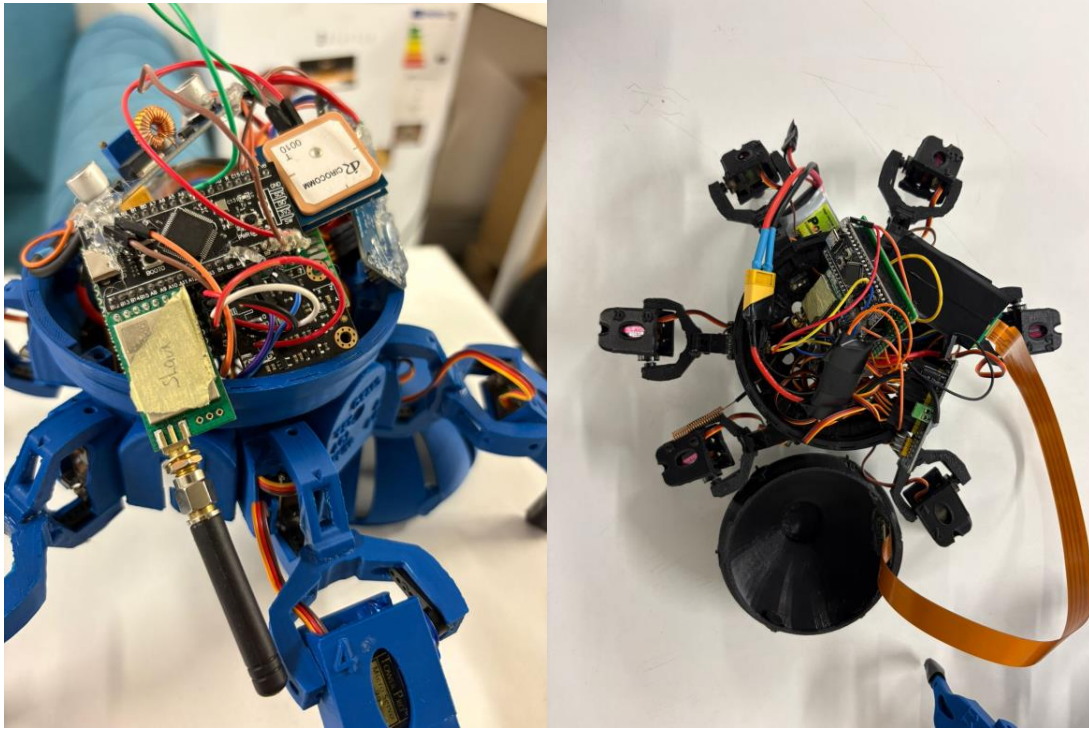


Figure 14

Master STM32 Firmware (main.c)

The Master firmware is structured around a non-blocking main loop with four concurrent tasks: command queue processing, LoRa packet handling, GPS parsing, and periodic telemetry transmission. All HAL_Delay calls and I2C operations are isolated in the main loop; the ISR performs only queue push and flag setting. Figure 7 shows the Class Diagram of firmware modules.

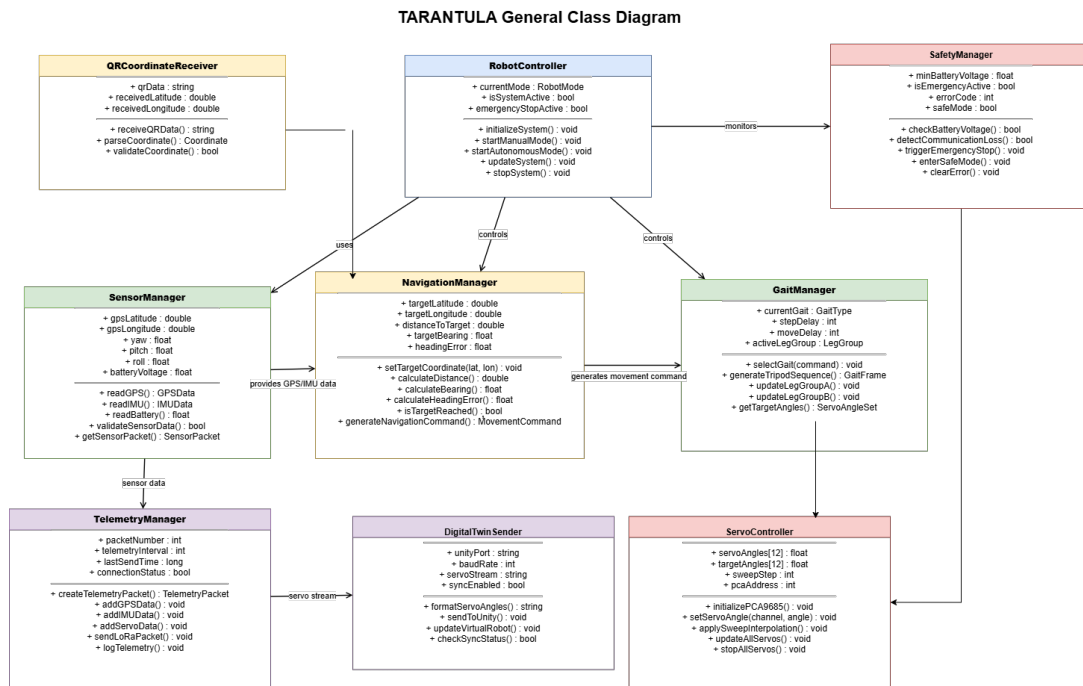


Figure 7: Class Diagram

USART Configuration

Table 5.1 USART Configuration

USART	Instance	TX/RX Pins	Baud Rate	Purpose
USART1	huart1	PA9 / PA10	9600	Raspberry Pi Zero 2W - QR GPS data (\$QR: packets)
USART2	huart2	PA2 / PA3	115200	BN-880 GPS - NMEA GGA sentence stream
USART6	huart6	PC6 / PC7	9600	Ebyte E220-900T22D LoRa - addressed packets

Non-Blocking Command Queue Architecture

A circular buffer of `CMD_QUEUE_SIZE=4` bytes implements command queuing. The UART6 ISR pushes single uppercase ASCII command bytes (F/B/L/R/S/G/A) without any `HAL_Delay` or I2C operations. The main loop pops and dispatches commands only when `loco_busy==0`. On step completion, `loco_busy` is reset to 0 and `$M:ACK` is sent to GS. This architecture resolves the critical bug where locomotion `HAL_Delay` calls in the ISR froze the system after the first command.

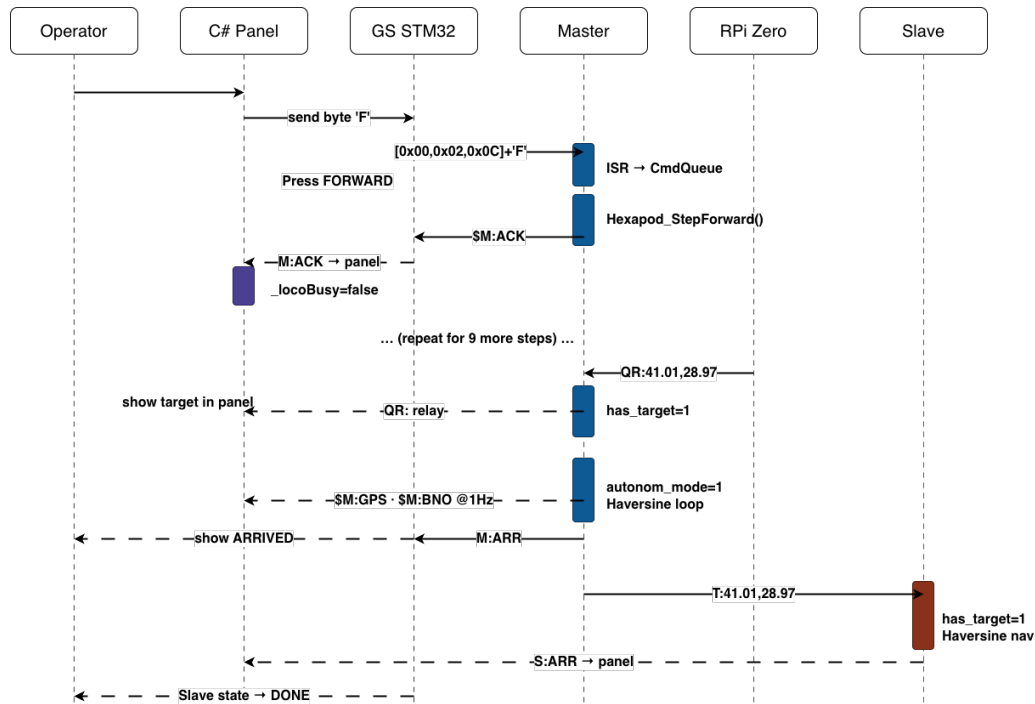


Figure 8: Sequence Diagram - Full coordination scenario message sequence

BNO055 Stability Fix

The original firmware polled OPR_MODE register on every telemetry cycle and triggered BNO055_Init() if the mode appeared incorrect, creating a reinit loop that zeroed Euler registers continuously. The fix removes mode polling entirely. BNO055_Read() directly reads the 6-byte Euler register block (BNO055_EULER_H_L = 0x1A) with a 50 ms timeout. A fail_count counter triggers hardware I2C recovery (HAL_I2C_DeInit -> 10 ms -> MX_I2C1_Init -> 10 ms -> BNO055_Init) only after 3 consecutive HAL_ERROR returns. A zero-guard preserves the last valid reading if all three raw values are simultaneously zero.

locomotion.h - Tripod Gait Library

The locomotion library implements two-phase tripod gait. Group A (legs 0, 2, 4: FR, RR, ML) and Group B (legs 1, 3, 5: MR, FL, RL) alternate between swing and push phases. Phase 1: Group A lifts (Femur -> FEMUR_LIFT) then swings forward (Coxa -> COXA_FORWARD); Group B pushes backward. Phase 2: Group B lifts and swings; Group A pushes. Each phase includes SETTLE_MS (120 ms) and PHASE_MS (180 ms) delays totalling approximately 600 ms per complete step.

Table 5.2 Leg Channel Mapping

Leg	Position	Coxa CH	Femur CH
Leg 0	Front-Right (FR)	CH0	CH1
Leg 1	Mid-Right (MR)	CH2	CH3
Leg 2	Rear-Right (RR)	CH4	CH5
Leg 3	Front-Left (FL)	CH6	CH7

Leg	Position	Coxa CH	Femur CH
Leg 4	Mid-Left (ML)	CH8	CH9
Leg 5	Rear-Left (RL)	CH10	CH11

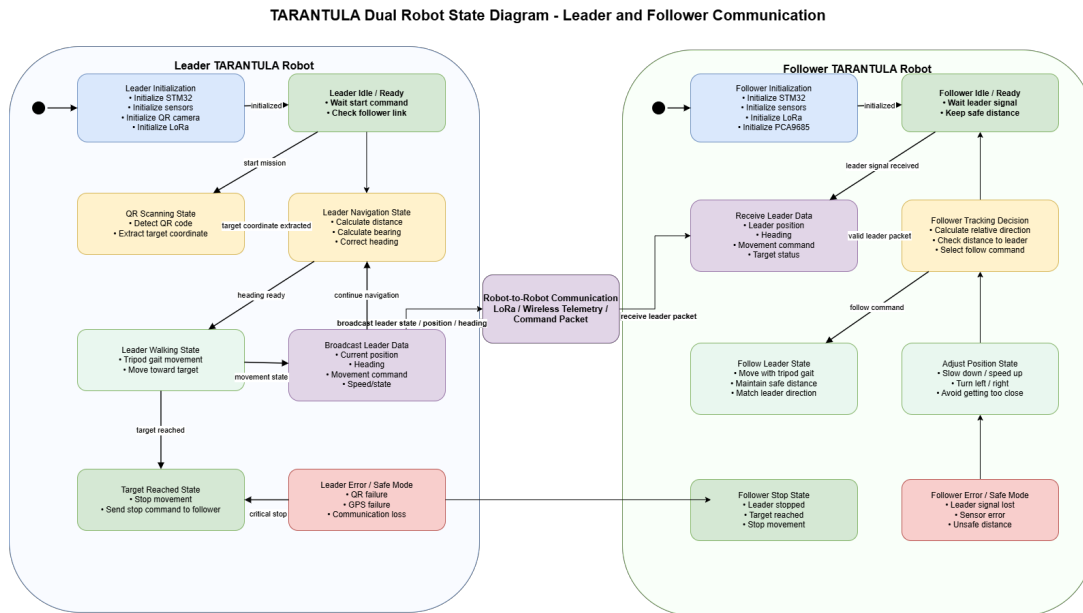


Figure 9: State Diagram - Master and Slave robot state machines

Activity Diagram - UML Command Flow

The Activity Diagram (Figure 10) shows the complete command flow from button press to servo movement across all swimlanes: Operator/C#, GS STM32, Master STM32, and Slave STM32. It illustrates the non-blocking queue dispatch, ACK return path, and coordination trigger.

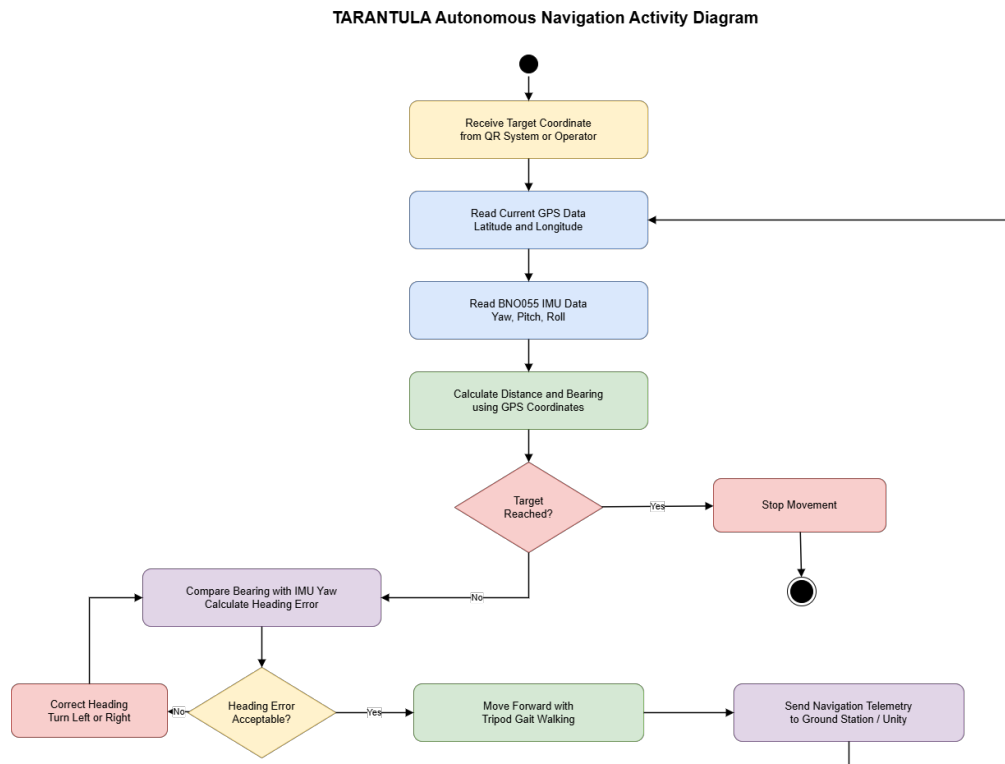


Figure 10(a): Autonomous Navigation Activity Diagram

TARANTULA Manual Locomotion Activity Diagram

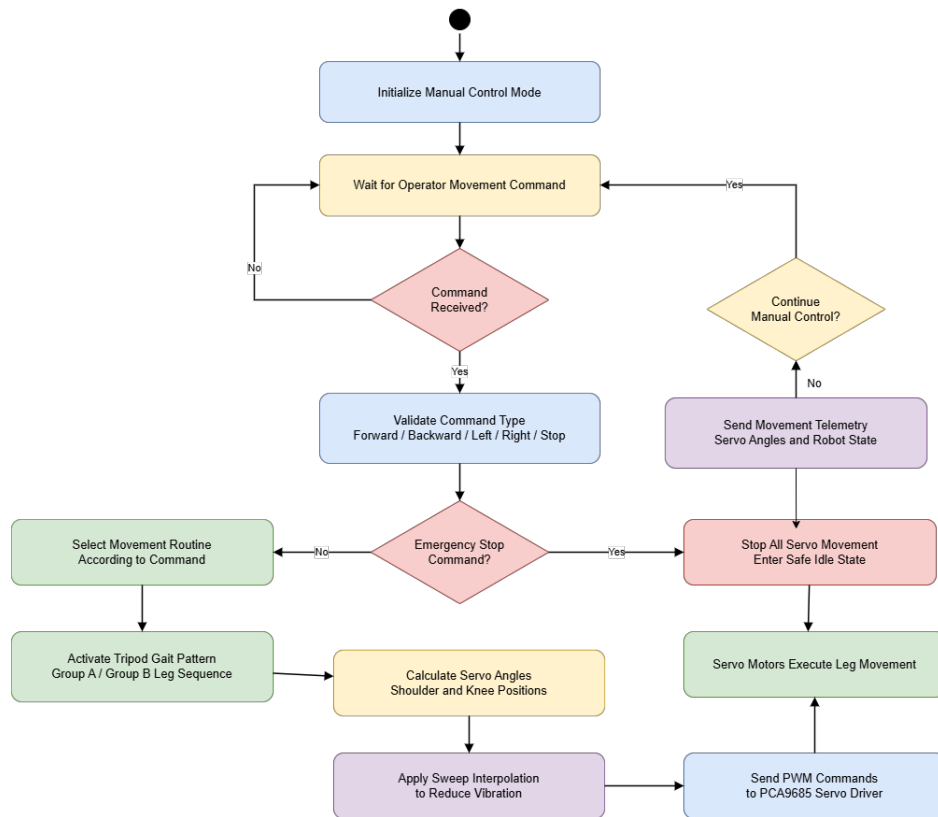


Figure 10(b): Manual Locomotion Activity Diagram

C# WPF Control Panel (TarantulaControl)

The TarantulaControl application is built on .NET 8 WPF using System.IO.Ports NuGet package. Features: CTRL PORT selector for GS COM port at 9600 baud; D-pad (F/B/L/R/S) with W/A/S/D keyboard shortcuts; step counter (0-10) and progress bar; Slave state indicator (IDLE/WALK/DONE); GPS panel for both Master and Slave (lat, lon, alt, satellites, fix status); BNO055 panel with compass needle animation (yaw/pitch/roll) and calibration bars; timestamped log with Save/Open; ACK flow control (_locoBusy flag) with 3-second watchdog timer.

Raspberry Pi Zero 2W - QR Perception Script

The QR detection script uses `picamera2.Picamera2()` for camera access and `pyzbar.decode()` for QR decoding. Expected payload format: lat,lon (e.g., 41.012345,28.974321). On successful decode, `pyserial` transmits `$QR:{lat},{lon}\n` to Master via `/dev/serial0` at 9600 baud. A 5-second cooldown prevents duplicate transmissions. Connect RPi GPIO 14 (TX) to Master PA10 (USART1 RX) with common GND.

In addition to QR packet transmission, the updated perception requirement specifies that the Raspberry Pi Zero 2W hosts a Flask-based HTTP server for real-time camera streaming. OpenCV-processed frames are displayed with bounding boxes around detected QR codes so that the operator can visually confirm target detection from the C# WPF panel. The local detected_qrs.txt state log is

used to prevent repeated transmission of the same target coordinate and to reduce USART1 pipeline congestion.

Unity 3D Digital Twin Visualization

The Unity Digital Twin module provides virtual representation of the Tarantula hexapod robot in a campus-style 3D environment. Key components: TarantulaRobotModel (hexapod mesh), CampusBuilding (university building with entrance, windows, lighting), CameraFollowController (drone-style smooth follow with adjustable height, backward distance, look-ahead, and rotation offset), and DigitalTwinManager (prepared for future serial or TCP real-time data input). The module does not control servo motors or replace STM32 firmware; it acts as a visualization and demonstration layer.

Developer Guide for Future Work

STM32 Firmware Installation:

- Open STM32CubeIDE. Create new STM32F401CCU6 project.
- In CubeMX: Enable I2C1 (Master/Slave only), USART1/2/6 (Master) or USART2/6 (Slave/GS), GPIO PC13 output.
- Copy locomotion.h to Core/Inc/. Copy main.c content within USER CODE blocks.
- Build and flash via Run -> Run. Verify [MASTER/SLAVE/GS] Ready messages on Serial Monitor at 115200 baud.

C# WPF Application:

- Open TarantulaControl.csproj in Visual Studio 2022. Build -> Build Solution (System.IO.Ports NuGet auto-restores).
- Run -> Start. Select GS COM port. Click CONNECT. LED indicator turns green.

Raspberry Pi QR Script:

- Enable camera: `sudo raspi-config -> Interface -> Camera -> Enable`.
- Install dependencies: `sudo apt install python3-pyzbar python3-picamera2 python3-serial`
- Connect RPi GPIO 14 (TX) -> Master PA10 (USART1 RX). Common GND. Run: `python3 qr_reader.py`

Known Limitations and Future Work:

- LoRa E220 AT command configuration must be done manually before deployment (no auto-config at boot).
- BNO055 requires figure-8 calibration motion each power cycle (no NVRAM persistent calibration).
- No obstacle avoidance - system operates on open flat terrain.
- Unity Digital Twin TCP real-time synchronization is planned but not yet implemented.
- Future: multi-target mission sequencing, terrain-adaptive gait switching, dynamic LoRa channel selection.

6 Other Supporting Information

This section provides supplementary technical information that supports understanding of the Tarantula Hexapod Robot System.

Table 6.1 LoRa Packet Format Summary

Packet Type	Format	Source	Destination
GPS Telemetry	\$M:GPS,lat,lon,alt,sats,fix\n	Master	GS -> C# Panel
IMU Telemetry	\$M:BNO,yaw,pitch,roll,sys,gyr,acc,mag\n	Master	GS -> C# Panel
Slave GPS	\$S:GPS,lat,lon,alt,sats,fix\n	Slave -> Master relay	GS -> C# Panel
Slave BNO	\$S:BNO,yaw,pitch,roll,...\n	Slave -> Master relay	GS -> C# Panel
QR Target	\$QR:lat,lon\n	RPi -> Master	Master -> GS -> C# Panel
Target Dispatch	\$T:lat,lon\n	Master	Slave
Command	Single ASCII byte (F/B/L/R/S/G/A)	C# -> GS	Master
Acknowledgement	\$M:ACK\n	Master	GS -> C# Panel
Arrival	\$M:ARR\n	Master	GS -> C# Panel
Warning	\$M:WARN,message\n	Master	GS -> C# Panel

Sample Firmware Code Blocks

The following excerpt illustrates the non-blocking command queue pop logic in the main loop:

```
if (!loco_busy) { uint8_t cmd = 0; if (CmdQueue_Pop(&cmd)) { switch (cmd) { case 'F':
Hexapod_StepForward(); break; case 'B': Hexapod_StepBackward(); break; ... }
RelayToGS("$M:ACK\n", 7); } }
```

The following excerpt illustrates the autonomous navigation decision logic:

```
double dist = Haversine(my_lat, my_lon, target_lat, target_lon); double bearing =
Bearing(my_lat, my_lon, target_lat, target_lon); double yaw_err = bearing -
(double)bno_yaw; // normalize to [-180, 180] if (dist < ARRIVED_DIST_M) { autonom_mode =
0; Hexapod_Stop(); } else if (yaw_err > YAW_THRESHOLD) Hexapod_TurnLeft(); else if
(yaw_err < -YAW_THRESHOLD) Hexapod_TurnRight(); else Hexapod_StepForward();
```

Project Deployment and Running Guide

This section explains how to deploy and run the Tarantula Hexapod Robot System from scratch on the required devices.

Required Installations and Tools

The project requires STM32CubeIDE for opening, building, and flashing STM32 firmware. Visual Studio 2022 and .NET 8 are required for running the C# WPF TarantulaControl interface. Unity Editor is required for opening and running the Unity 3D Digital Twin project. Raspberry Pi OS, Python 3.x, and the required camera, QR detection, and serial communication libraries are required for the Raspberry Pi QR detection module. The LoRa modules must also be configured with compatible address, channel, and baud rate settings before system operation.

STM32 Firmware Setup

First, open the STM32 firmware project in STM32CubeIDE. Check that the required peripherals are configured correctly, including UART, I2C, GPIO, GPS, BNO055, PCA9685, and LoRa communication settings. Build the project and connect the STM32 board to the computer. Flash the firmware to the STM32 board and verify that the firmware starts correctly. The Master STM32 must be able to process movement commands, read GPS and BNO055 data, communicate with the Raspberry Pi through UART, and exchange packets with the Ground Station and Slave robot through LoRa.

C# WPF Control Panel Setup

Open the TarantulaControl project in Visual Studio 2022. Restore the required NuGet packages if needed and build the solution. Run the application and select the correct Ground Station COM port. The serial connection should use 9600 baud. After pressing the CONNECT button, the interface should show an active connection status. The interface can then send movement commands, display telemetry, show robot state information, save logs, send TCP commands to Unity, and display the Unity camera stream.

Unity Digital Twin Setup

Open the Unity project using Unity Editor and open the main Unity scene. Press Play to start the Unity Digital Twin module. The Unity scene contains the virtual Tarantula robot model and the campus-style environment. Unity listens for TCP movement commands from the C# WPF interface. If Unity and the C# interface run on the same computer, the Unity TCP address can be set as 127.0.0.1:7777. If Unity runs on another computer on the same network, the IPv4 address of the Unity computer must be used with port 7777. The Unity View stream can also be opened from the C# WPF interface to observe the Unity camera output.

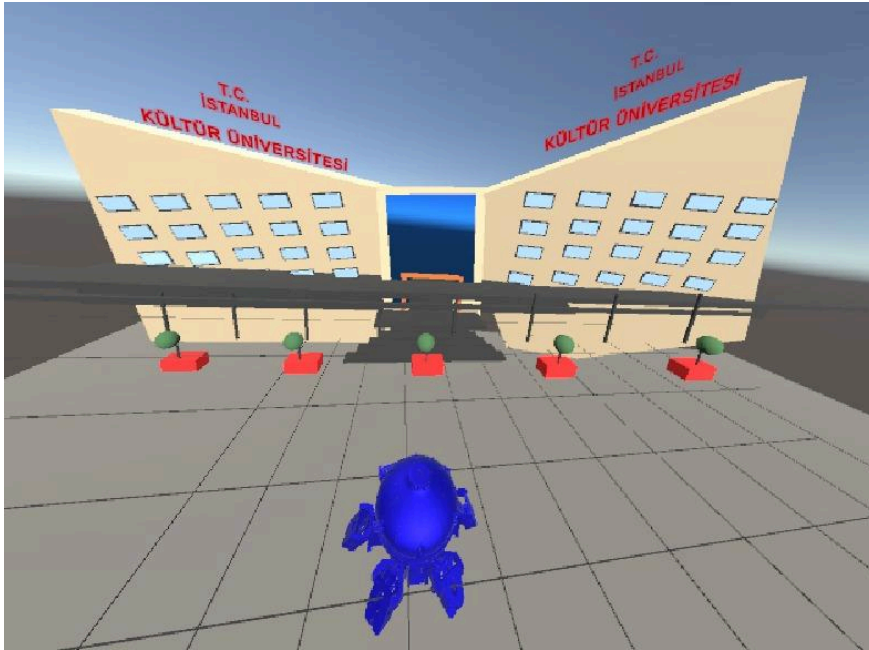


Figure 14: Unity Digital Twin of the Tarantula robot in the campus-style simulation environment.

Raspberry Pi QR Module Setup

Enable the Raspberry Pi camera and install the required Python libraries for camera access, QR detection, and serial communication. Connect the Raspberry Pi UART TX pin to the Master STM32 UART RX pin and connect common GND. Run the QR detection Python script on the Raspberry Pi. When a QR code containing latitude and longitude values is detected, the Raspberry Pi sends the target coordinate message to the Master STM32 through UART.

LoRa Communication Setup

The LoRa modules must be configured with the correct address, channel, baud rate, and communication mode. The Ground Station, Master robot, and Slave robot must use compatible LoRa settings. After configuration, the Ground Station should be able to receive telemetry messages from the Master and Slave robots and forward them to the C# WPF interface. The Master should also be able to forward target information and commands to the Slave robot.

Running the Full System

To run the full system, power the robot hardware and start the STM32 firmware. Then start the C# WPF control panel and connect to the Ground Station COM port. Start the Unity scene and connect the Unity TCP address from the interface. If QR target detection is required, start the Raspberry Pi QR script. After all modules are active, test manual movement commands first. Then verify telemetry display, BNO055 orientation values, GPS status, LoRa communication, Unity visualization, Unity View stream, and emergency stop behavior. Autonomous mode should only be started when target data, GPS fix, and IMU readiness conditions are satisfied.

Environment Variables, Database, and Login Credentials

No database connection is required for this project. No login credentials are required. No environment variables are required for the current implementation. The main configuration values are COM port selection, baud rate, LoRa settings, Unity TCP address, Unity stream address, and Raspberry Pi serial port configuration.

Backup and Alternative Demo Scenario

Before presentation or submission, the full project folder should be backed up on USB and cloud storage. If the physical robot or wireless communication fails during the presentation, the C# WPF interface and Unity Digital Twin can be used as an alternative demonstration scenario to show movement commands, Unity visualization, local-direction movement, and the Unity camera stream inside the interface.

Initial Design Decisions vs Final Design Decisions

This section compares the main design decisions made at the beginning of the project with their updated or improved final versions.

Decision 1: Robot Control Architecture

Before:

The system was planned as a manually controlled hexapod robot with basic movement commands.

After:

The final system includes manual movement, autonomous navigation logic, QR-based target acquisition, GPS and BNO055 sensor processing, LoRa communication, and digital twin visualization.

Reason:

The design was expanded to create a more complete robotic system that demonstrates movement, sensing, communication, navigation, and visualization together.

Decision 2: Movement Command Processing

Before:

Movement commands were planned to be handled directly when received.

After:

A non-blocking command queue structure was used. Commands are processed in the main loop, and the system uses acknowledgement messages and watchdog behavior to prevent command lock-up.

Reason:

This change improves reliability and prevents the robot firmware from freezing during movement command execution.

Decision 3: Sensor-Based Navigation

Before:

The robot was expected to move using basic directional commands.

After:

GPS and BNO055 data were integrated into the navigation process. The robot can calculate distance and bearing to a target coordinate and use yaw information for direction correction.

Reason:

Sensor-based navigation was required to support autonomous movement toward a target.

Decision 4: Target Acquisition

Before:

Target coordinates could have been manually defined or fixed in the software.

After:

A Raspberry Pi camera-based QR system was added to provide dynamic target coordinates. The Raspberry Pi reads QR data and sends latitude and longitude values to the STM32.

Reason:

This makes the system more flexible because target coordinates can be changed without modifying firmware code.

Decision 5: Wireless Communication

Before:

The system could operate with direct or local communication only.

After:

LoRa communication was added between the Ground Station, Master robot, and Slave robot. Telemetry, target data, commands, warnings, acknowledgement messages, and arrival messages are transferred through LoRa.

Reason:

LoRa communication allows the system to demonstrate wireless telemetry and multi-robot coordination.

Decision 6: Master-Slave Coordination

Before:

The project could have focused only on one robot.

After:

A Master-Slave coordination structure was included. The Master robot can receive QR target data and forward target information to the Slave robot.

Reason:

This design improves the project scope by adding coordinated robot behavior and distributed communication.

Decision 7: C# WPF Control Panel

Before:

The interface was planned mainly for sending movement commands and displaying basic data.

After:

The final C# WPF interface supports serial connection, movement commands, telemetry display, robot state monitoring, logging, TCP communication with Unity, and Unity camera stream display.

Reason:

The interface was improved to become the main monitoring and control point of the system.

Decision 8: Unity Digital Twin Communication

Before:

The Unity Digital Twin module was initially planned mainly as a visualization environment and was prepared for possible future serial or TCP-based data input.

After:

The Unity module was connected to the C# WPF interface through TCP. The interface can send movement commands such as forward, backward, left, right, and stop to the Unity Digital Twin.

Reason:

This change allows the operator to control and observe the virtual robot directly from the same interface.

Decision 9: Unity Robot Movement Direction

Before:

The virtual robot movement was based on a fixed world direction. After turning, the forward command could still move the robot in the previous fixed direction.

After:

The movement logic was updated to use the robot's local facing direction. After the robot turns, the forward command moves it in the direction it is currently facing.

Reason:

This change makes the virtual robot movement more realistic and supports direction-based and diagonal movement.

Decision 10: Unity Camera Observation

Before:

The Unity scene could be observed mainly through the Unity Editor camera view.

After:

A drone-style camera follow system was adjusted to keep the robot visible during movement. The Unity camera output was also streamed and displayed inside the C# WPF interface through a dedicated Unity View panel.

Reason:

This change improves real-time observation and allows the operator to monitor the Unity Digital Twin from the control interface.

Decision 11: Demonstration and Testing Strategy

Before:

Testing was planned separately for hardware, firmware, communication, and visualization modules.

After:

The final testing strategy includes full system integration testing. Manual movement, telemetry, QR target acquisition, LoRa communication, autonomous behavior, emergency stop, C# WPF interface operation, and Unity visualization are tested together.

Reason:

Integrated testing is required to verify that all modules work together as a complete robotic system.

7 References

- [1] Bor, M., Roedig, U., Voigt, T., & Alonso, J. M. (2016). Do LoRa low-power wide-area networks scale? In MSWiM '16: Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (ss. 59–67). ACM Press. <https://doi.org/10.1145/2988287.2989163>
- [2] Bosch Sensortec. (2016). BNO055 intelligent 9-axis absolute orientation sensor (BST-BNO055-DS000-14). <https://www.bosch-sensortec.com>

-
- [3] Bosch Sensortec. (t.y.). BNO055 intelligent 9-axis absolute orientation sensor [Veri sayfası]. Adafruit. https://cdn-shop.adafruit.com/datasheets/BST_BNO055_DS000_12.pdf
- [4] Chengdu Ebyte Electronic Technology. (t.y.). Ürün veri sayfası (Belge ID: 1463) [Veri sayfası]. <https://www.cdebyte.com/pdf-down.aspx?id=1463>
- [5] Ebyte. (2022). E220-900T22D LoRa module user manual (V1.3). <https://www.ebyte.com>
- [6] Heimann, H. (t.y.). stm32-hal-pca9685 [Kaynak kodu]. GitHub. <https://github.com/henriheimann/stm32-hal-pca9685>
- [7] ivyknob. (t.y.). bno055_stm32 [Kaynak kodu]. GitHub. https://github.com/ivyknob/bno055_stm32
- [8] Kau, N., Schultz, A., Ferrante, N., & Slade, P. (2019). Stanford Doggo: An open-source, quasi-direct-drive quadruped. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA) (ss. 6309–6315). Montreal, QC, Canada. <https://doi.org/10.1109/ICRA.2019.8794436>
- [9] Microsoft. (2024). .NET 8 WPF documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>
- [10] NXP Semiconductors. (2015). PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED controller (Rev 4). <https://cdn-shop.adafruit.com/datasheets/PCA9685.pdf>
- [11] Raspberry Pi Foundation. (2023). Picamera2 library documentation. <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- [12] STMicroelectronics. (2015). STM32F401xB/C reference manual (RM0368, Rev 5). https://www.st.com/resource/en/reference_manual/rm0368-stm32f401xbc-stmicroelectronics.pdf
- [13] STMicroelectronics. (t.y.). STM32F401xD, STM32F401xE [Veri sayfası]. <https://www.st.com/resource/en/datasheet/stm32f401re.pdf>
- [14] Unity Technologies. (2024). Unity documentation. <https://docs.unity3d.com>
- [15] Yun, Y., Jin, J., Kim, N., Yoon, J., & Kim, C. (2012). Outdoor localization with optical navigation sensor, IMU and GPS. 2012 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI) (ss. 386–391). Hamburg, Germany. <https://doi.org/10.1109/MFI.2012.6343007>
- [16] Tedeschi, F., & Carbone, G. (2014). Design issues for hexapod walking robots. *Robotics*, 3(2), 181–206. <https://doi.org/10.3390/robotics3020181>
- [17] Spenko, M., Haynes, G. C., Saunders, J., Cutkosky, M., Rizzi, A. A., Full, R. J., & Koditschek, D. E. (2008). Biologically inspired climbing with a hexapedal robot. *Journal of Field Robotics*, 25(4–5), 223–242. <https://doi.org/10.1002/rob.20238>
- [18] Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2nd ed.). MIT Press.
- [19] Siciliano, B., Sciavicelli, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, planning and control*. Springer.

-
- [20] Kaplan, E. D., & Hegarty, C. J. (Eds.). (2017). *Understanding GPS/GNSS: Principles and applications* (3rd ed.). Artech House.
- [21] Borenstein, J., Everett, H. R., & Feng, L. (1996). *Navigating mobile robots: Systems and techniques*. A. K. Peters.
- [22] Campion, G., Bastin, G., & D'Andrea-Novet, B. (1996). Structural properties and classification of kinematic and dynamic models of wheeled mobile robots. *IEEE Transactions on Robotics and Automation*, 12(1), 47–62. <https://doi.org/10.1109/70.481750>
- [23] Zhu, Q., & Hu, J. (2010). Behaviour based motion control for mobile robots. *IET Control Theory & Applications*, 4(10), 2220–2230. <https://doi.org/10.1049/iet-cta.2009.0420>
- [24] Kau, N., Schultz, A., Ferrante, N., & Slade, P. (2019). Stanford Doggo: An open-source, quasi-direct-drive quadruped. In *Proceedings of the 2019 International Conference on Robotics and Automation (ICRA)* (pp. 6309–6315). <https://doi.org/10.1109/ICRA.2019.8794436>
- [25] Arm, P., Zenkl, R., Barton, P., Beyer, L., Dietsche, A., Löchner, U., Bhatt, H., Riner, R., Höpflinger, M. A., & Hutter, M. (2019). SpaceBok: A dynamic legged robot for space exploration. In *Proceedings of the 2019 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 6288–6294). <https://doi.org/10.1109/ICRA.2019.8794136>
- [26] Elmenreich, W. (2002). *Sensor fusion in time-triggered systems* [Doctoral dissertation]. Vienna University of Technology.
- [27] Caballero, F., Merino, L., Ferruz, J., & Ollero, A. (2009). Vision-based odometry and SLAM for medium and high altitude flying UAVs. *Journal of Intelligent and Robotic Systems*, 54(1–3), 137–161. <https://doi.org/10.1007/s10846-008-9257-8>
- [28] Ferreira, J. F., & Dias, J. (2014). *Probabilistic approach to robot perception*. Springer.
- [29] Yun, Y., Jin, J., Kim, N., Yoon, J., & Kim, C. (2012). Outdoor localization with optical navigation sensor, IMU and GPS. In *Proceedings of the 2012 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)* (pp. 386–391). <https://doi.org/10.1109/MFI.2012.6343007>
- [30] Tomic, T., Schmid, K., Lutz, P., Dömel, A., Kassecker, M., Mair, E., Grixa, I. L., Ruess, F., Suppa, M., & Burschka, D. (2012). Toward a fully autonomous UAV: Research platform for indoor and outdoor urban search and rescue. *IEEE Robotics & Automation Magazine*, 19(3), 46–56. <https://doi.org/10.1109/MRA.2012.2206473>
- [31] Raibert, M., Blankespoor, K., Nelson, G., & Playter, R. (2008). BigDog, the rough-terrain quadruped robot. In *Proceedings of the 17th IFAC World Congress* (pp. 10822–10825). <https://doi.org/10.3182/20080706-5-KR-1001.01833>
- [32] Saranli, U., Buehler, M., & Koditschek, D. E. (2001). RHex: A simple and highly mobile hexapod robot. *International Journal of Robotics Research*, 20(7), 616–631. <https://doi.org/10.1177/02783640122067570>

-
- [33] Niku, S. B. (2020). *Introduction to robotics: Analysis, control, applications* (3rd ed.). Wiley.
- [34] Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press.
- [35] Craig, J. J. (2005). *Introduction to robotics: Mechanics and control* (3rd ed.). Pearson Prentice Hall.
- [36] Latombe, J. C. (1991). *Robot motion planning*. Kluwer Academic Publishers.
- [37] LaValle, S. M. (2006). *Planning algorithms*. Cambridge University Press.
- [38] Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: An open-source Robot Operating System. In *ICRA Workshop on Open Source Software*, 3(3.2), 5.
- [39] Bräunl, T. (2008). *Embedded robotics: Mobile robot design and applications with embedded systems* (3rd ed.). Springer.
- [40] Wahde, M. (2008). *Biologically inspired optimization methods: An introduction*. WIT Press.
- [41] Full, R. J., & Koditschek, D. E. (1999). Templates and anchors: Neuromechanical hypotheses of legged locomotion on land. *Journal of Experimental Biology*, 202(23), 3325–3332. <https://doi.org/10.1242/jeb.202.23.3325>
- [42] Shill, J., Collins, E., Coyle, E., & Dunlap, J. (2015). Requirements for a man-portable military robot. In *Proceedings of the 2015 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)* (pp. 1–6). <https://doi.org/10.1109/SSRR.2015.7443011>
- [43] Vidoni, R., Gasparetto, A., & Giovagnoni, M. (2010). Design and implementation of an ERLS-based 3-D dynamic formulation for flexible-link robots. *Robotics and Computer-Integrated Manufacturing*, 26(4), 433–443. <https://doi.org/10.1016/j.rcim.2010.01.001>
- [44] Alatise, M. B., & Hancke, G. P. (2017). Pose estimation of a mobile robot based on fusion of IMU data and vision data using an extended Kalman filter. *Sensors*, 17(10), 2164. <https://doi.org/10.3390/s17102164>
- [45] Grzonka, S., Grisetti, G., & Burgard, W. (2012). A fully autonomous indoor quadrotor. *IEEE Transactions on Robotics*, 28(1), 90–100. <https://doi.org/10.1109/TRO.2011.2162999>
- [46] Madgwick, S. O. H., Harrison, A. J. L., & Vaidyanathan, R. (2011). Estimation of IMU and MARG orientation using a gradient descent algorithm. In *Proceedings of the 2011 IEEE International Conference on Rehabilitation Robotics (ICORR)* (pp. 1–7). <https://doi.org/10.1109/ICORR.2011.5975346>
- [47] Titterton, D. H., & Weston, J. L. (2004). *Strapdown inertial navigation technology* (2nd ed.). IET.

- [48] Gezici, S. (2008). A survey on wireless position estimation. *Wireless Personal Communications*, 44(3), 263–282. <https://doi.org/10.1007/s11277-007-9375-z>
- [49] Censi, A. (2008). An ICP variant using a point-to-line metric. In *Proceedings of the 2008 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 19–25). <https://doi.org/10.1109/ICRA.2008.4543181>
- [50] Zumberge, J. F., Heflin, M. B., Jefferson, D. C., Watkins, M. M., & Webb, F. H. (1997). Precise point positioning for the efficient and robust analysis of GPS data from large networks. *Journal of Geophysical Research: Solid Earth*, 102(B3), 5005–5017. <https://doi.org/10.1029/96JB03860>
- [51] Sola, J. (2017). *Quaternion kinematics for the error-state Kalman filter* [Technical report]. LAAS-CNRS. <https://arxiv.org/abs/1711.02508>
- [52] Stachniss, C., Leonard, J. J., & Thrun, S. (2016). Simultaneous localization and mapping. In B. Siciliano & O. Khatib (Eds.), *Springer handbook of robotics* (2nd ed., pp. 1153–1176). Springer. https://doi.org/10.1007/978-3-319-32552-1_46
- [53] Bor, M., Roedig, U., Voigt, T., & Alonso, J. M. (2016). Do LoRa low-power wide-area networks scale? In *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)* (pp. 59–67). <https://doi.org/10.1145/2988287.2989163>
- [54] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., & Watteyne, T. (2017). Understanding the limits of LoRaWAN. *IEEE Communications Magazine*, 55(9), 34–40. <https://doi.org/10.1109/MCOM.2017.1600613>
- [55] Semtech Corporation. (2015). *SX1276/77/78/79 datasheet: 137 MHz to 1020 MHz low power long range transceiver* (Rev 7). <https://www.semtech.com/uploads/documents/sx1276.pdf>
- [56] Augustin, A., Yi, J., Clausen, T., & Townsley, W. M. (2016). A study of LoRa: Long range and low power networks for the Internet of Things. *Sensors*, 16(9), 1466. <https://doi.org/10.3390/s16091466>
- [57] Townsend, C., & Arms, S. (2005). Wireless sensor networks: Principles and applications. In J. G. Webster (Ed.), *Sensor technology handbook* (pp. 575–589). Elsevier.
- [58] Winkler, A. W., Bellicoso, C. D., Hutter, M., & Buchli, J. (2018). Gait and trajectory optimization for legged systems through phase-based end-effector parameterization. *IEEE Robotics and Automation Letters*, 3(3), 1560–1567. <https://doi.org/10.1109/LRA.2018.2798285>
- [59] Asfour, T., Regenstein, K., Azad, P., Schroder, J., Bierbaum, A., Vahrenkamp, N., & Dillmann, R. (2006). ARMAR-III: An integrated humanoid platform for sensory-motor control. In *Proceedings of the 2006 IEEE-RAS International Conference on Humanoid Robots* (pp. 169–175). <https://doi.org/10.1109/ICHR.2006.321380>

- [60] Metta, G., Sandini, G., Vernon, D., Natale, L., & Nori, F. (2008). The iCub humanoid robot: An open platform for research in embodied cognition. In *Proceedings of the 8th Workshop on Performance Metrics for Intelligent Systems* (pp. 50–56). <https://doi.org/10.1145/1774674.1774683>
- [61] Ijspeert, A. J. (2008). Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4), 642–653. <https://doi.org/10.1016/j.neunet.2008.03.014>
- [62] Kimura, H., Fukuoka, Y., & Cohen, A. H. (2007). Adaptive dynamic walking of a quadruped robot on natural ground based on biological concepts. *International Journal of Robotics Research*, 26(5), 475–490. <https://doi.org/10.1177/0278364907078089>
- [63] Hardarson, F. (1997). *Locomotion for difficult terrain* [Doctoral dissertation]. Royal Institute of Technology (KTH).
- [64] Selig, J. M. (2004). *Geometrical methods in robotics*. Springer.
- [65] Chirikjian, G. S., & Burdick, J. W. (1995). The kinematics of hyper-redundant robot locomotion. *IEEE Transactions on Robotics and Automation*, 11(6), 781–793. <https://doi.org/10.1109/70.478426>
- [66] Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5), 1147–1163. <https://doi.org/10.1109/TRO.2015.2463671>
- [67] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2), 91–110. <https://doi.org/10.1023/B:VISI.0000029664.99615.94>
- [68] Bradski, G. (2000). The OpenCV library. *Dr. Dobbs's Journal of Software Tools*, 25(11), 120–125.
- [69] Hartley, R., & Zisserman, A. (2004). *Multiple view geometry in computer vision* (2nd ed.). Cambridge University Press.
- [70] Torr, P. H. S., & Zisserman, A. (2000). MLESAC: A new robust estimator with application to estimating image geometry. *Computer Vision and Image Understanding*, 78(1), 138–156. <https://doi.org/10.1006/cviu.1999.0832>
- [71] Fischler, M. A., & Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381–395. <https://doi.org/10.1145/358669.358692>
- [72] Nister, D., & Stewenius, H. (2006). Scalable recognition with a vocabulary tree. In *Proceedings of the 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)* (Vol. 2, pp. 2161–2168). <https://doi.org/10.1109/CVPR.2006.264>
- [73] Szeliski, R. (2010). *Computer vision: Algorithms and applications*. Springer.
- [74] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

[75] Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.